

- 全面指导Linux内核的设计与实现
- 填补Linux内核理论与实践之间的鸿沟
- 畅销技术图书新版

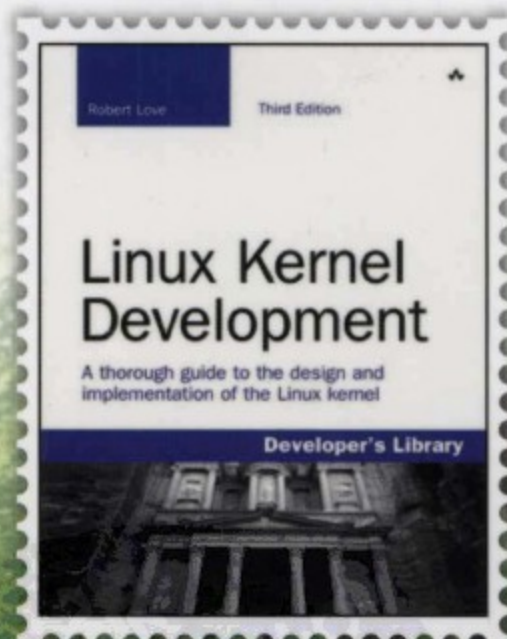
原书第3版

Linux

内核设计与实现

Linux Kernel Development Third Edition

(美) Robert Love 著
陈莉君 康 华 译



华章专业开发者书库

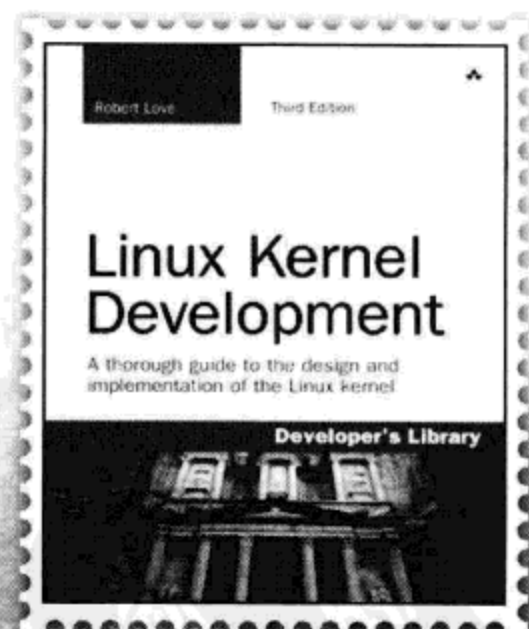
原书第3版

Linux

内核设计与实现

Linux Kernel Development Third Edition

(美) Robert Love 著
陈莉君 康华 译



机械工业出版社
China Machine Press

本书基于 Linux 2.6.34 内核详细介绍了 Linux 内核系统, 覆盖了从核心内核系统的应用到内核设计与实现等各方面的内容。本书主要内容包括: 进程管理、进程调度、时间管理和定时器、系统调用接口、内存寻址、内存管理和页缓存、VFS、内核同步以及调试技术等。同时本书也涵盖了 Linux 2.6 内核中颇具特色的内容, 包括 CFS 调度程序、抢占式内核、块 I/O 层以及 I/O 调度程序等。本书采用理论与实践相结合的路线, 能够带领读者快速走进 Linux 内核世界, 真正开发内核代码。

本书适合作为高等院校操作系统课程的教材或参考书, 也可供相关技术人员参考。

Simplified Chinese edition copyright © 2011 by Pearson Education Asia Limited and China Machine Press.

Original English language title: *Linux Kernel Development Third Edition* (ISBN 978-0-672-32946-3) by Robert Love, Copyright © 2010.

All rights reserved.

Published by arrangement with the original publisher, Pearson Education, Inc., publishing as Addison-Wesley.

本书封面贴有 Pearson Education (培生教育出版集团) 激光防伪标签, 无标签者不得销售。

封底无防伪标均为盗版

版权所有, 侵权必究

本书法律顾问 北京市展达律师事务所

本书版权登记号: 图字: 01-2010-4824

图书在版编目 (CIP) 数据

Linux 内核设计与实现 (原书第 3 版) / (美) 拉芙 (Love, R) 著; 陈莉君, 康华译. —北京: 机械工业出版社, 2011.6

(华章专业开发者书库)

书名原文: *Linux Kernel Development, Third Edition*

ISBN 978-7-111-33829-1

I. L… II. ①拉… ②陈… ③康… III. Linux 操作系统—程序设计 IV. TP316.89

中国版本图书馆 CIP 数据核字 (2011) 第 047916 号

机械工业出版社 (北京市西城区百万庄大街 22 号 邮政编码 100037)

责任编辑: 秦 健

北京市荣盛彩色印刷有限公司印刷

2011 年 6 月第 1 版第 1 次印刷

186mm×240mm·22 印张

标准书号: ISBN 978-7-111-33829-1

定价: 69.00 元

凡购本书, 如有缺页、倒页、脱页, 由本社发行部调换

客服热线: (010) 88378991; 88361066

购书热线: (010) 68326294; 88376949; 68995259

投稿热线: (010) 88379604

读者信箱: hzjsj@hzbook.com



译者序

不知不觉涉足 Linux 内核已经十多个年头了，与其他有志（兴趣）于此的朋友一样，我们也经历了学习—实用—追踪—再学习的过程。也就是说，我们也是从漫无边际到茫然无措，再到初窥门径，转而觉得心有戚戚焉这一路走下来的。其中甘苦，犹然在心。

Linux 最为人称道的莫过于它的自由精神，所有源代码唾手可得。侯捷先生云：“源码在前，了无秘密。”是的，但是我们在面对它的时候，为什么却总是因为这种规模和层面所造就的陡峭学习曲线陷入困顿呢？很多朋友就此倒下，纵然 Linux 世界繁花似锦，纵然内核天空无边广阔，但是，眼前的迷雾重重，心中的阴霾又怎能被阳光驱散呢？纵有雄心壮志，拔剑四顾心茫然，脚下路在何方？

Linux 内核入门是不容易，它之所以难学，在于庞大的规模和复杂的层面。规模一大，就不易现出本来面目，浑然一体，自然不容易找到着手之处；层面一多，就会让人眼花缭乱，盘根错节，怎能让人提纲挈领？

“如果有这样一本书，既能提纲挈领，为我理顺思绪，指引方向，同时又能照顾小节，阐述细微，帮助我们更好更快地理解 STL 源码，那该有多好。”孟岩先生如此说，虽然针对的是 C++，但道出的是研习源码的人们共同的心声。然而，Linux 源码研究的方法却不大相同。这还是由于规模和层面决定的。比如说，在语言学习中，我们可以采取小步快跑的方法，通过一个个小程序和小尝试，就可以取得渐进的成果，就能从新技术中有所收获。而掌握 Linux 呢？如果没有对整体的把握，即使你对某个局部的算法、技术或是代码再熟悉，也无法将其融入实用。其实，像内核这样的大规模的软件，正是编程技术施展身手的舞台（当然，目前的内核虽然包含了一些面向对象思想，但还不能让 C++ 一展身手）。

那么，我们能不能做点什么，让 Linux 的内核学习过程更符合程序员的习惯呢？

Robert Love 回答了这个问题。Robert Love 是一个狂热的内核爱好者，所以他的想法自然贴近程序员。是的，我们注定要在对所有核心的子系统有了全面认识之后，才能开始自己的实践，但却完全可以舍弃细枝末节，将行李压缩到最小，自然可以轻装快走，迅速进入动手阶段。

因此，相对于 Daniel P. Bovet 和 Marco Cesati 的内核巨著《Understanding the Linux Kernel》，它少了五分细节；相对于实践经典《Linux Device Drivers》，多了五分说理。可以说，本书填补了 Linux 内核理论和实践之间的鸿沟，真可谓“一桥飞架南北，天堑变通途”。

就我们的经验，内核初学者（不是编程初学者）可以从本书着手，对内核各个核心子系统有个整体把握，包括它们提供什么样的服务，为什么要提供这样的服务，又是怎样实现的。而且，本书还包含了 Linux 内核开发者在开发时需要用到的很多信息，包括调试技术、编程风格、注意事项等。在消化本书的基础上，如果你侧重于了解内核，可以进一步研究《Understanding the Linux Kernel》和源代码本身；如果你侧重于实际编程，可以研读《Linux Device Drivers》，

直接开始动手工作；如果你想有一个轻松的内核学习和实践环节，请访问我们的网站 www.kerneltravel.net。

Linux 内核是一艘永不停息的轮船，它将驶向何方我们并不知晓，但在这些变化的背后，总有一些原理是恒定不变的，总有一些变化是我们想知晓的，比如调度程序的大幅度改进，内核性能的不断提升，本书第 3 版虽然针对的是较新的 2.6.34 Linux 内核版本，但在旧版本上积累的知识 and 经验依然有效，而新增内容将使读者在应对变化了的内核代码时更加从容。

感谢牛涛和武特，他们在第 2 版和第 3 版差异的校对中花费了大量精力。感谢素不相识的网友 Cheng Renquan，他主动承担了其中一章的修订。还要感谢苏锦绣、黄伟、王泽宇、赵格娟、刘周平、周永飞、曹江峰、陈白虎和孟阿龙，他们参与了后期的校对和查错补漏。

最后，特别感谢我的合作者康华，从十年前一块分析 Linux 内核代码到今天，他对技术孜孜不倦的追求不但在业界赢得声誉，也使我们在翻译过程中所遇到的技术难点和晦涩句子被一一迎刃而解。感谢合作者张波，他流畅有趣的文笔让本书少了份枯燥，多了份趣味。

陈莉君

2010 年 10 月



序 言

随着 Linux 内核和 Linux 应用程序越来越成熟，越来越多的系统软件工程师涉足 Linux 开发和维护领域。他们中有些人纯粹是出于个人爱好，有些人是为 Linux 公司工作，有些人是为硬件厂商做开发，还有一些是为内部项目工作的。

但是所有人都必须直面一个问题：内核的学习曲线变得越来越长，也越来越陡峭。系统规模不断扩大，复杂程度不断提高。虽然现在的内核开发者对内核的掌握越发炉火纯青，但新手却无法跟上内核发展的步伐，长此以往将出现青黄不接的断层。

我认为这种新老鸿沟已经成为内核质量的一个隐患，而且问题将继续恶化。所以那些真正关心内核的人已经开始致力于扩大内核开发群体。

解决上述问题的一个方法是尽量保证代码简洁：接口定义合理，代码风格一致，“一次做一件事，做到完美”等。这也就是 Linus Torvalds 倡导的解决办法。

我提倡的解决办法是对代码慷慨地加上注释，即能够让读者立刻了解代码开发者意图的文字（识别意图和实现之间差异的工作称为调试。如果意图不明确显然调试就难以进行）。

可是，即使有注解，也没办法清楚地展现内核的各个主要子系统的全景，说明它们到底要做什么。那么，这些开发者又该从何下手呢？

由文字材料来说明这些在起步阶段就该理解的材料，其实是最合适的。

Robert Love 的贡献就在于此，有经验的开发者可以通过本书全面了解内核子系统提供的服务，同时还可以了解这些服务是怎么实现的。对不少人来说，这些知识就已经足够了：那些好奇的人，那些应用程序开发者，那些想对内核的设计品头论足一番的人，都有足够的谈资了。

但是学习本书同样可以作为那些有抱负的内核开发者更上一层楼的契机，可以帮他们更改内核代码以达到预定的目标。我建议有抱负的开发者能够亲身实践：理解内核某部分的捷径就是对它做些修改，这样能为开发者揭示仅仅通过看内核代码无法看到的深层机理。

严肃认真的内核开发者应该加入开发邮件列表，不断和其他开发者交流。这是内核开发者相互切磋和并肩前进的最好方法。而 Robert 在书中对内核生活中至关重要的文化和技巧都做了精彩介绍。

请学习和欣赏 Robert 的书吧。想必你也希望能精益求精，继续探索，成为内核开发社区中的一员，那么首先你要清楚的是：社区欢迎你。我们评价和衡量一个人是根据他所作的贡献，当你投身于 Linux 时，你要明白：虽然你仅仅贡献了一小份力，但马上就会有数千万或上亿人受益。这是我们的欢乐之源，也是我们的责任之本。

Andrew Morton

前言

在我刚开始有把自己的内核开发经验集结成册，撰写一本书的念头时，我其实也觉得有点头绪繁多，不知道该从何下手。我实在不想落入传统内核书籍的窠臼，照猫画虎地再写这么一本。不错，前人著述备矣，但我终归是要写出点儿与众不同的东西来，我的书该如何定位，说实话，这确实让人颇费思量。

后来，灵感终于浮现出来，我意识到自己可以从一个全新的视角看待这个主题。开发内核是我的工作，同时也是我的嗜好，内核就是我的挚爱。这些年来，我不断搜集与内核有关的奇闻轶事，不断积攒关键的开发诀窍，依靠这些日积月累的材料，我可以写一本关于开发内核该做什么——更重要的是——不该做什么的书籍。从本质上说，这本书仍旧是描述 Linux 内核是如何设计和实现的，但是写法却另辟蹊径，所提供的信息更倾向于实用。通过本书，你就可以做一些内核开发的工作了——并且是使用正确的方法去做。我是一个注重实效的人，因此，这是一本实践的书，它应当有趣、易读且有用。

我希望读者可以从这本书中领略到更多 Linux 内核的精妙之处（写出来的和没写出来的），也希望读者敢于从阅读本书和读内核代码开始跨越到开始尝试开发可用、可靠且清晰的内核代码。当然如果你仅仅是兴致所至，读书自娱，那也希望你能从中找到乐趣。

从第 1 版到现在，又过了一段时间，我们再次回到本书，修补遗憾。本版比第 1 版和第 2 版内容更丰富：修订、补充并增加了新的内容和章节，使其更加完善。本版融合了第 2 版以来内核的各种变化。更值得一提的是，Linux 内核联盟^①做出决定，近期内不进行 2.7 版内核的开发，于是，内核开发者打算继续开发并稳定 2.6 版。这个决定意味深长，而本书从中的最大受益就是在 2.6 版上可以稳定相当长时间。随着内核的成熟，内核“快照”才有机会能维持得更久远一些。本书可作为内核开发的规范文档，既认识内核的过去，也着眼于内核的未来。

使用这本书

开发 Linux 内核不需要天赋异秉，不需要有什么魔法，连 Unix 开发者普遍长着的络腮胡子都不一定要有。内核虽然有一些有趣并且独特的规则和要求，但是它和其他大型软件项目相比，并没有太大差别。像所有的大型软件开发一样，要学的东西确实不少，但是不同之处在于数量上的积累，而非本质上的区别。

认真阅读源码非常有必要，Linux 系统代码的开放性其实是弥足珍贵的，不要无动于衷地将它搁置一边，浪费了大好资源。实际上就是读了代码还远远不够呢，你应该钻研并尝试着动手改动一些代码。寻找一个 bug 然后去修改它，改进你的硬件设备的驱动程序。增加新功能，即使看

^① 这一决定是在加拿大渥太华 2004 年夏季举办的 Linux 内核年度开发者大会上做出的。

起来微不足道，寻找痛痒之处并解决。只有动手写代码才能真正融会贯通。

内核版本

本书基于 Linux 2.6 内核系列。它并不涵盖早期的版本，当然也有一些例外。比如，我们会讨论 2.4 系列内核中的一些子系统是如何实现的，这是因为简单的实现有助于传授知识。特别说明的是，本书介绍的是最新的 Linux 2.6.34 内核版本。尽管内核总在不断更新，任何努力也难以捕获这样一只永不停息的猛兽，但是本书力图适合于新旧内核的开发者和用户。

虽然本书讨论的是 2.6.34 内核，但我也确保了它同样适用于 2.6.32 内核。后一个版本往往被各个 Linux 发行版本奉为“企业版”内核，所以我们可以各种产品线上见到其身影。该版本确实已经开发了数年（类似的“长线”版本还有 2.6.9、2.6.18 和 2.6.27 等）。

读者范围

本书是写给那些有志于理解 Linux 内核的软件开发者的。本书并不逐行逐字地注解内核源代码，也不是指导开发驱动程序或是内核 API 的参考手册（如果存在标准的内核 API 的话）。本书的初衷是提供足够多的关于 Linux 内核设计和实现的信息，希望读过本书的程序员能够拥有较为完备的知识，可以真正开始开发内核代码。无论开发内核是为了兴趣还是为了赚钱，我都希望能够带领读者快速走进 Linux 内核世界。本书不但介绍了理论而且也讨论了具体应用，可以满足不同读者的需要。全书紧紧围绕着理论联系实践，并非一味强调理论或是实践。无论你研究 Linux 内核的动机是什么，我都希望这本书都能将内核的设计和实现分析清楚，起到抛砖引玉的作用。

因此，本书覆盖了从核心内核系统的应用到内核设计与实现等各方面的内容。我认为这点很重要，值得花工夫讨论。例如，第 8 章讨论的是所谓的下半部机制。本章分别讨论了内核下半部机制的设计和实现（核心内核开发者或者学者会感兴趣），随即便介绍了如何使用内核提供的接口实现你自己的下半部（这对设备驱动开发者可能很有用处）。其实，我认为上述两部分内容是相得益彰的，虽然核心内核开发者主要关注的问题是内核内部如何工作，但是也应该清楚如何使用接口；同样，如果设备驱动开发者了解了接口背后的实现机制，自然也会受益匪浅。

这好比学习某些库的 API 函数与研究该库的具体实现。初看，好像应用程序开发者仅仅需要理解 API——我们被灌输的思想是，应该像看待黑盒子一样看待接口。另外，库的开发者也只关心库的设计与实现，但是我认为双方都应该花时间相互学习。能深刻了解操作系统本质的应用程序开发者无疑可以更好地利用它。同样，库开发者也决不应该脱离基于此库的应用程序，埋头开发。因此，我既讨论了内核子系统的设计，也讨论了它的用法，希望本书能对核心开发者和应用开发者都有用。

我假设读者已经掌握了 C 语言，而且对 Linux 比较熟悉。如果读者还具有与操作系统设计相关的经验和其他计算机科学的概念就更好了。当然，我也会尽可能多地解释这些概念，但如果你仍然不能理解这些知识的话，请看本书最后参考资料中给出的一些关于操作系统设计方面的经典书籍。

本书很适合在大学中作为介绍操作系统的辅助教材，与介绍操作系统理论的书相搭配。对于

大学高年级课程或者研究生课程来说，可直接使用本书作为教材。

第 3 版致谢

与其他作者一样，我决非是一个人躲在山洞里孤苦地写出这本书来的（那也是一件美差，因为有熊相伴）。我能最终完成本书原稿是与无数建议和关怀分不开的。仅仅一页纸无法容纳我的感激，但我还是要衷心地感谢所有给予我鼓励、给予我知识和给予我灵感的朋友和同事。

首先我要对为此书付出辛勤劳作的所有编辑表示感谢，尤其要感谢我的组稿编辑 Mark Taber，他为这一版的出版从头到尾倾注了许多心血。还要特别感谢业务开发编辑 Michael Thurston 和项目组织编辑 Tonya Simpso。

本版的技术编辑 Robert P. J. Day 也是我需要倍加感谢的人，他独到的洞察力和准确的校对使书稿质量大大提高。尽管他的工作称得上完美，但如果书中仍留有错误，责任由我承担。需要十分感谢的还有 Adam Belay、Zack Brown、Martin Pool 以及 Chris Rivera，他们对第 1 版和第 2 版所做的一切努力我依然记忆犹新。

许多内核开发者为我提供了大力支持，回答了许多问题，还有那些撰写代码的人——本书正是由于有了这些代码，才有了存在的意义。他们是：Andrea Arcangeli、Alan Cox、Greg Kroah-Hartman、Dave Miller、Patrick Mochel、Andrew Morton、Nick Piggin 和 Linus Torvalds。

我要大力感谢我的同事们。他们的创造力和智慧无与伦比，能与他们一起工作其乐无穷。因为篇幅原因，请谅解我不能列出所有人的名字。但不得不提的是：Alan Blount、Jay Crim、Chris Danis、Chris DiBona、Eric Flatt、Mike Lockwood、San Mehat、Brian Rogan、Brian Swetland、Jon Trowbridge 和 Steve Vinter，感谢他们给予我的支持、知识和友谊！

还有许多值得尊敬和热爱的人，他们是：Paul Amici、Mikey Babbitt、Keith Barbag、Jacob Berkman、Nat Friedman、Dustin Hall、Joyce Hawkins、Miguel de Icaza、Jimmy Krehl、Doris Love、Linda Love、Brette Luck、Randy O'Dowd、Sal Ribaud 和他了不起的妈妈 Chris Rivera、Carolyn Rodon、Joey Shaw、Sarah Stewart、Jeremy VanDoren 和他的家人、Luis Villa、Steve Weisberg 和他的家人以及 Helen Whisnant 等。

最后，非常感谢我的父母。

内核黑客万岁！

Robert Love
Boston

PDG

作者简介

Robert Love 是一位资深的开源软件开发者、讲师和作者，他使用 Linux 和贡献于 Linux 已超过 15 年。目前他是 Google 公司的资深软件工程师，是 Android 移动平台内核开发团队的成员；在去 Google 工作之前，他就职于 Novell 公司，任职 Linux 桌面系统的首席架构师；在去 Novell 之前，他是 MontaVista 和 Ximain 公司的内核开发工程师。

Robert 参与的内核项目包括抢占式内核、进程调度器、内核事件层、通知机制、VM 改进，以及一些设备驱动。

Robert 曾经发表过许多关于 Linux 内核的演讲和文章；他还是《Linux journal》电子杂志的编辑。另外，除了本书，他的著作还包含《Linux System Programming》和《Linux in a Nutshell》。

Robert 在佛罗里达大学获得数学学士学位和计算机理学学士学位。



目 录

译者序
序 言
前 言
作者简介

第 1 章 Linux 内核简介	1
1.1 Unix 的历史	1
1.2 追寻 Linus 足迹：Linux 简介	2
1.3 操作系统和内核简介	3
1.4 Linux 内核和传统 Unix 内核的比较 ...	5
1.5 Linux 内核版本	7
1.6 Linux 内核开发者社区	8
1.7 小结	8
第 2 章 从内核出发	10
2.1 获取内核源码	10
2.1.1 使用 Git	10
2.1.1 安装内核源代码	10
2.1.3 使用补丁	11
2.2 内核源码树	11
2.3 编译内核	12
2.3.1 配置内核	12
2.3.2 减少编译的垃圾信息	14
2.3.3 衍生多个编译作业	14
2.3.4 安装新内核	14
2.4 内核开发的特点	15
2.4.1 无 libc 库抑或无标准头文件	15
2.4.2 GNU C	16
2.4.3 没有内存保护机制	18

2.4.4 不要轻易在内核中使用浮点数 ...	18
2.4.5 容积小而固定的栈	18
2.4.6 同步和并发	18
2.4.7 可移植性的重要性	19
2.5 小结	19
第 3 章 进程管理	20
3.1 进程	20
3.2 进程描述符及任务结构	21
3.2.1 分配进程描述符	22
3.2.2 进程描述符的存放	23
3.2.3 进程状态	23
3.2.4 设置当前进程状态	25
3.2.5 进程上下文	25
3.2.6 进程家族树	25
3.3 进程创建	26
3.3.1 写时拷贝	27
3.3.2 fork()	27
3.3.3 vfork()	28
3.4 线程在 Linux 中的实现	28
3.4.1 创建线程	29
3.4.2 内核线程	30
3.5 进程终结	31
3.5.1 删除进程描述符	32
3.5.2 孤儿进程造成的进退维谷	32
3.6 小结	34
第 4 章 进程调度	35
4.1 多任务	35
4.2 Linux 的进程调度	36

4.3 策略	36	5.4.2 参数传递	60
4.3.1 I/O 消耗型和处理器消耗型的 进程	36	5.5 系统调用的实现	61
4.3.2 进程优先级	37	5.5.1 实现系统调用	61
4.3.3 时间片	38	5.5.2 参数验证	62
4.3.4 调度策略的活动	38	5.6 系统调用上下文	64
4.4 Linux 调度算法	39	5.6.1 绑定一个系统调用的最后步骤	65
4.4.1 调度器类	39	5.6.2 从用户空间访问系统调用	67
4.4.2 Unix 系统中的进程调度	40	5.6.3 为什么不通过系统调用的 方式实现	68
4.4.3 公平调度	41	5.7 小结	68
4.5 Linux 调度的实现	42	第 6 章 内核数据结构	69
4.5.1 时间记账	42	6.1 链表	69
4.5.2 进程选择	44	6.1.1 单向链表和双向链表	69
4.5.3 调度器入口	48	6.1.2 环形链表	70
4.5.4 睡眠和唤醒	49	6.1.3 沿链表移动	71
4.6 抢占和上下文切换	51	6.1.4 Linux 内核中的实现	71
4.6.1 用户抢占	53	6.1.5 操作链表	73
4.6.2 内核抢占	53	6.1.6 遍历链表	75
4.7 实时调度策略	54	6.2 队列	78
4.8 与调度相关的系统调用	54	6.2.1 kfifo	79
4.8.1 与调度策略和优先级相关的 系统调用	55	6.2.2 创建队列	79
4.8.2 与处理器绑定有关的系统调用	55	6.2.3 推入队列数据	79
4.8.3 放弃处理器时间	56	6.2.4 摘取队列数据	80
4.9 小结	56	6.2.5 获取队列长度	80
第 5 章 系统调用	57	6.2.6 重置和撤销队列	80
5.1 与内核通信	57	6.2.7 队列使用举例	81
5.2 API、POSIX 和 C 库	57	6.3 映射	81
5.3 系统调用	58	6.3.1 初始化一个 idr	82
5.3.1 系统调用号	59	6.3.2 分配一个新的 UID	82
5.3.2 系统调用的性能	59	6.3.3 查找 UID	83
5.4 系统调用处理程序	60	6.3.4 删除 UID	84
5.4.1 指定恰当的系统调用	60	6.3.5 撤销 idr	84
		6.4 二叉树	84

6.4.1 二叉搜索树	84	8.2 软中断	110
6.4.2 自平衡二叉搜索树	85	8.2.1 软中断的实现	111
6.5 数据结构以及选择	87	8.2.2 使用软中断	113
6.6 算法复杂度	88	8.3 tasklet	114
6.6.1 算法	88	8.3.1 tasklet 的实现	114
6.6.2 大 o 符号	88	8.3.2 使用 tasklet	116
6.6.3 大 θ 符号	89	8.3.3 老的 BH 机制	119
6.6.4 时间复杂度	89	8.4 工作队列	120
6.7 小结	90	8.4.1 工作队列的实现	121
第 7 章 中断和中断处理	91	8.4.2 使用工作队列	124
7.1 中断	91	8.4.3 老的任务队列机制	126
7.2 中断处理程序	92	8.5 下半部机制的选择	127
7.3 上半部与下半部的对比	93	8.6 在下半部之间加锁	128
7.4 注册中断处理程序	93	8.7 禁止下半部	128
7.4.1 中断处理程序标志	94	8.8 小结	129
7.4.2 一个中断例子	95	第 9 章 内核同步介绍	131
7.4.3 释放中断处理程序	95	9.1 临界区和竞争条件	131
7.5 编写中断处理程序	96	9.1.1 为什么我们需要保护	132
7.5.1 共享的中断处理程序	97	9.1.2 单个变量	133
7.5.2 中断处理程序实例	97	9.2 加锁	134
7.6 中断上下文	99	9.2.1 造成并发执行的原因	135
7.7 中断处理机制的实现	100	9.2.2 了解要保护些什么	136
7.8 /proc/interrupts	102	9.3 死锁	137
7.9 中断控制	103	9.4 争用和扩展性	138
7.9.1 禁止和激活中断	103	9.5 小结	140
7.9.2 禁止指定中断线	105	第 10 章 内核同步方法	141
7.9.3 中断系统的状态	105	10.1 原子操作	141
7.10 小结	106	10.1.1 原子整数操作	142
第 8 章 下半部和推后执行的		10.1.2 64 位原子操作	144
工作	107	10.1.3 原子位操作	145
8.1 下半部	107	10.2 自旋锁	147
8.1.1 为什么要用下半部	108	10.2.1 自旋锁方法	148
8.1.2 下半部的环境	108	10.2.2 其他针对自旋锁的操作	149

10.2.3 自旋锁和下半部	150	11.7.2 定时器竞争条件	180
10.3 读-写自旋锁	150	11.7.3 实现定时器	180
10.4 信号量	152	11.8 延迟执行	181
10.4.1 计数信号量和二值信号量	153	11.8.1 忙等待	181
10.4.2 创建和初始化信号量	154	11.8.2 短延迟	182
10.4.3 使用信号量	154	11.8.3 <code>schedule_timeout()</code>	183
10.5 读-写信号量	155	11.9 小结	185
10.6 互斥体	156	第 12 章 内存管理	186
10.6.1 信号量和互斥体	158	12.1 页	186
10.6.2 自旋锁和互斥体	158	12.2 区	187
10.7 完成变量	158	12.3 获得页	189
10.8 BLK: 大内核锁	159	12.3.1 获得填充为 0 的页	190
10.9 顺序锁	160	12.3.2 释放页	191
10.10 禁止抢占	161	12.4 <code>kmalloc()</code>	191
10.11 顺序和屏障	162	12.4.1 <code>gfp_mask</code> 标志	192
10.12 小结	165	12.4.2 <code>kfree()</code>	195
第 11 章 定时器和时间管理	166	12.5 <code>vmalloc()</code>	196
11.1 内核中的时间概念	166	12.6 slab 层	197
11.2 节拍率: HZ	167	12.6.1 slab 层的设计	198
11.2.1 理想的 HZ 值	168	12.6.2 slab 分配器的接口	200
11.2.2 高 HZ 的优势	169	12.7 在栈上的静态分配	203
11.2.3 高 HZ 的劣势	169	12.7.1 单页内核栈	203
11.3 jiffies	170	12.7.2 在栈上光明正大地工作	203
11.3.1 jiffies 的内部表示	171	12.8 高端内存的映射	204
11.3.2 jiffies 的回绕	172	12.8.1 永久映射	204
11.3.3 用户空间和 HZ	173	12.8.2 临时映射	204
11.4 硬时钟和定时器	174	12.9 每个 CPU 的分配	205
11.4.1 实时时钟	174	12.10 新的每个 CPU 接口	206
11.4.2 系统定时器	174	12.10.1 编译时的每个 CPU 数据	206
11.5 时钟中断处理程序	174	12.10.2 运行时的每个 CPU 数据	207
11.6 实际时间	176	12.11 使用每个 CPU 数据的原因	208
11.7 定时器	178	12.12 分配函数的选择	209
11.7.1 使用定时器	178	12.13 小结	209

第 13 章 虚拟文件系统	210	14.5.7 I/O 调度程序的选择	245
13.1 通用文件系统接口	210	14.6 小结	246
13.2 文件系统抽象层	211	第 15 章 进程地址空间	247
13.3 Unix 文件系统	212	15.1 地址空间	247
13.4 VFS 对象及其数据结构	213	15.2 内存描述符	248
13.5 超级块对象	214	15.2.1 分配内存描述符	249
13.6 超级块操作	215	15.2.2 撤销内存描述符	250
13.7 索引节点对象	217	15.2.3 mm_struct 与内核线程	250
13.8 索引节点操作	219	15.3 虚拟内存区域	251
13.9 目录项对象	222	15.3.1 VMA 标志	251
13.9.1 目录项状态	222	15.3.2 VMA 操作	253
13.9.2 目录项缓存	223	15.3.3 内存区域的树型结构和内存 区域的链表结构	254
13.10 目录项操作	224	15.3.4 实际使用中的内存区域	254
13.11 文件对象	225	15.4 操作内存区域	255
13.12 文件操作	226	15.4.1 find_vma()	256
13.13 和文件系统相关的数据结构	230	15.4.2 find_vma_prev()	257
13.14 和进程相关的数据结构	232	15.4.3 find_vma_intersection()	257
13.15 小结	233	15.5 mmap() 和 do_mmap() : 创建地址 区间	258
第 14 章 块 I/O 层	234	15.6 mmap() 和 do_mmap() : 删除 地址区间	259
14.1 剖析一个块设备	234	15.7 页表	260
14.2 缓冲区和缓冲区头	235	15.8 小结	261
14.3 bio 结构体	237	第 16 章 页高速缓存和页回写	262
14.3.1 I/O 向量	238	16.1 缓存手段	262
14.3.2 新老方法对比	239	16.1.1 写缓存	262
14.4 请求队列	240	16.1.2 缓存回收	263
14.5 I/O 调度程序	240	16.2 Linux 页高速缓存	264
14.5.1 I/O 调度程序的工作	241	16.2.1 address_space 对象	264
14.5.2 Linus 电梯	241	16.2.2 address_space 操作	266
14.5.3 最终期限 I/O 调度程序	242	16.2.3 基树	267
14.5.4 预测 I/O 调度程序	244	16.2.4 以前的页散列表	268
14.5.5 完全公正的排队 I/O 调度 程序	244		
14.5.6 空操作的 I/O 调度程序	245		

16.3 缓冲区高速缓存	268	18.2 内核中的 bug	296
16.4 flusher 线程	268	18.3 通过打印来调试	296
16.4.1 膝上型计算机模式	270	18.3.1 健壮性	296
16.4.2 历史上的 bdfush、kupdated 和 pdflush	270	18.3.2 日志等级	297
16.4.3 避免堵塞的方法：使用多线程 ..	271	18.3.3 记录缓冲区	298
16.5 小结	271	18.3.4 syslogd 和 klogd	298
第 17 章 设备与模块	273	18.3.5 从 printf() 到 printk() 的转换 ..	298
17.1 设备类型	273	18.4 oops	298
17.2 模块	274	18.4.1 ksymoops	300
17.2.1 Hello, World	274	18.4.2 kallsyms	300
17.2.2 构建模块	275	18.5 内核调试配置选项	301
17.2.3 安装模块	277	18.6 引发 bug 并打印信息	301
17.2.4 产生模块依赖性	277	18.7 神奇的系统请求键	302
17.2.5 载入模块	278	18.8 内核调试器的传奇	303
17.2.6 管理配置选项	279	18.8.1 gdb	303
17.2.7 模块参数	280	18.8.2 kgdb	304
17.2.8 导出符号表	282	18.9 探测系统	304
17.3 设备模型	283	18.9.1 用 UID 作为选择条件	304
17.3.1 kobject	283	18.9.2 使用条件变量	305
17.3.2 ktype	284	18.9.3 使用统计量	305
17.3.3 kset	285	18.9.4 重复频率限制	305
17.3.4 kobject、ktype 和 kset 的 相互关系	285	18.10 用二分查找法找出引发罪恶的 变更	306
17.3.5 管理和操作 kobject	286	18.11 使用 Git 进行二分搜索	307
17.3.6 引用计数	287	18.12 当所有的努力都失败时：社区 ..	308
17.4 sysfs	288	18.13 小结	308
17.4.1 sysfs 中添加和删除 kobject ..	290	第 19 章 可移植性	309
17.4.2 向 sysfs 中添加文件	291	19.1 可移植操作系统	309
17.4.3 内核事件层	293	19.2 Linux 移植史	310
17.5 小结	294	19.3 字长和数据类型	311
第 18 章 调试	295	19.3.1 不透明类型	313
18.1 准备开始	295	19.3.2 指定数据类型	314
		19.3.3 长度明确的类型	314

19.3.4 char 型的符号问题	315	20.2.4 花括号	325
19.4 数据对齐	315	20.2.5 每行代码的长度	326
19.4.1 避免对齐引发的问题	316	20.2.6 命名规范	326
19.4.2 非标准类型的对齐	316	20.2.7 函数	326
19.4.3 结构体填补	316	20.2.8 注释	326
19.5 字节顺序	318	20.2.9 typedef	327
19.6 时间	319	20.2.10 多用现成的东西	328
19.7 页长度	320	20.2.11 在源码中减少使用 ifdef	328
19.8 处理器排序	320	20.2.12 结构初始化	328
19.9 SMP、内核抢占、高端 内存	321	20.2.13 代码的事后修正	329
19.10 小结	321	20.3 管理系统	329
第 20 章 补丁、开发和社区	322	20.4 提交错误报告	329
20.1 社区	322	20.5 补丁	330
20.2 Linux 编码风格	322	20.5.1 创建补丁	330
20.2.1 缩进	323	20.5.2 用 Git 创建补丁	331
20.2.2 switch 语句	323	20.5.3 提交补丁	331
20.2.3 空格	324	20.6 小结	332
		参考资料	333



第 ① 章

Linux 内核简介

第 1 章将带我们从 Unix 的历史视角来认识 Linux 内核与 Linux 操作系统的前世今生。今天 Unix 系统业已演化成一个具有相似应用程序编程接口 (API)，并且基于相似设计理念的操作系统家族。但它又是一个别具特色的操作系统，从萌芽到现在已经有 40 余年的历史。若要了解 Linux，我们必须首先认识 Unix 系统。

1.1 Unix 的历史

Unix 虽然已经使用了 40 年，但计算机科学家仍然认为它是现存操作系统中最强大和最优秀的系统。从 1969 年诞生以来，由 Dennis Ritchie 和 Ken Thompson 的灵感火花点亮的这个 Unix 产物已经成为一种传奇，它历经了时间的考验依然声名不坠。

Unix 是从贝尔试验室的一个失败的多用户操作系统 Multics 中涅槃而生的。Multics 项目被终止后，贝尔实验室计算科学研究中心的人们发现自己处于一个没有交互式操作系统可用的境地。在这种情况下，1969 年的夏天，贝尔实验室的程序员们设计了一个文件系统原型，而这个原型最终发展演化成了 Unix。Thompson 首先在一台无人问津的 PDP-7 型机上实现了这个全新的操作系统。1971 年，Unix 被移植到 PDP-11 型机中。1973 年，整个 Unix 操作系统用 C 语言进行了重写，正是当时这个并不太引人注目的举动，给后来 Unix 系统的广泛移植铺平了道路。第一个在贝尔实验室以外被广泛使用的 Unix 版本是第 6 版，称为 V6。

许多其他的公司也把 Unix 移植到新的机型上。伴随着这些移植，开发者们按照自己的方式不断地增强系统的功能，并由此产生了若干变体。1977 年，贝尔实验室综合各种变体推出了 Unix System III；1983 年 AT&T 推出了 System V[⊖]。

由于 Unix 系统设计简洁并且在发布时提供源代码，所以许多其他组织和团体都对它进行了进一步的开发。加州大学伯克利分校便是其中影响最大的一个。他们推出的变体叫 Berkeley Software Distributions (BSD)。伯克利的第一个 Unix 演化版是 1977 年推出的 1BSD 系统，它的实现基于贝尔实验室的 Unix 版本，不但在其上加入了许多修正补丁，而且还集成了不少额外的软件；1978 年伯克利继续推出了 2BSD 系统，其中包含我们如今仍在使用的 csh、vi 等应用软件。而伯克利真正独立开发的 Unix 系统是于 1979 年推出的 3BSD 系统，该系统引入了一系列令人振奋的新特性，支持虚拟内存便是其一大亮点。在 3BSD 以后，伯克利又相继推出了 4BSD 系列，包括 4.0BSD、4.1BSD、4.2BSD、4.3BSD 等众多分支。这些 Unix 演化版实现了任务管理、

⊖ 什么是系统 IV 呢？它是一个内部开发版本。

换页机制、TCP/IP 等新特性。最终伯克利大学在 1994 年重写了虚拟内存子系统 (VM)，并推出了伯克利 Unix 系统的最终官方版，即我们熟知的 4.4BSD。现在，多亏了 BSD 的开放性许可，BSD 的开发才得以由 Darwin、FreeBSD、NetBSD 和 OpenBSD 继续。

20 世纪 80 和 90 年代，许多工作站和服务器厂商推出了他们自己的 Unix，这些 Unix 大部分是在 AT&T 或伯克利发行版的基础上加上一些满足他们特定体系结构需要的特性。这其中就包括 Digital 的 Tru64、HP 的 HP-UX、IBM 的 AIX、Sequent 的 DYNIX/ptx、SGI 的 IRIX 和 Sun 的 Solaris 和 SunOS。

由于最初一流的设计和以后多年的创新与逐步提高，Unix 系统成为一个强大、健壮和稳定的操作系统。下面的几个特点是使 Unix 强大的根本原因。首先，Unix 很简洁：不像其他动辄提供数千个系统调用并且设计目的不明确系统，Unix 仅提供几百个系统调用并且有一个非常明确的设计目的。第二，在 Unix 中，所有的东西都被当做文件对待^①。这种抽象使对数据和对设备的操作是通过一套相同的系统调用接口来进行的：`open()`、`read()`、`write()`、`lseek()` 和 `close()`。第三，Unix 的内核和相关的系统工具软件是用 C 语言编写而成——正是这个特点使得 Unix 在各种硬件体系架构面前都具备令人惊异的移植能力，并且使广大的开发人员很容易就能接受它。第四，Unix 的进程创建非常迅速，并且有一个非常独特的 `fork()` 系统调用。最后，Unix 提供了一套非常简单但又很稳定的进程间通信元语，快速简洁的进程创建过程使 Unix 的程序把目标放在一次执行保质保量地完成一个任务上，而简单稳定的进程间通信机制又可以保证这些单一目的的简单程序可以方便地组合在一起，去解决现实中变得越来越复杂的任务。正是由于这种策略和机制分离的设计理念，确保了 Unix 系统具备清晰的层次化结构。

今天，Unix 已经发展成为一个支持抢占式多任务、多线程、虚拟内存、换页、动态链接和 TCP/IP 网络的现代化操作系统。Unix 的不同变体被应用在大到数百个 CPU 的集群，小到嵌入式设备的各种系统上。尽管 Unix 已经不再被认为是一个实验室项目了，但它仍然伴随着操作系统设计技术的进步而继续成长，人们仍然可以把它作为一个通用的操作系统来使用。

Unix 的成功归功于其简洁和一流的设计。它能拥有今天的能力和成就应该归功于 Dennis Ritchie、Ken Thompson 和其他早期设计人员的最初决策，同时也要归功于那些永不妥协于成见，从而赋予 Unix 无穷活力的设计抉择。

1.2 追寻 Linux 足迹：Linux 简介

1991 年，Linus Torvalds 为当时新推出的、使用 Intel 80386 微处理器的计算机开发了一款全新的操作系统，Linux 由此诞生。那时，作为芬兰赫尔辛基大学的一名学生的 Linus，正为不能随心所欲使用强大而自由的 Unix 系统而苦恼。对 Torvalds 而言，使用当时流行的 Microsoft 的 DOS 系统，除了玩波斯王子游戏外，别无他用。Linus 热衷使用于 Minix，一种教学用的廉价 Unix，但是，他不能轻易修改和发布该系统的源代码（由于 Minix 的许可证），也不能对 Minix 开发者所作的设计轻举妄动，这让他耿耿于怀并由此对作者的设计理念感到失望。

① 好吧，我承认，不是所有——但是确实很多东西都被表示成文件。Sockets 就是一个典型的例外。不过最近有不少尝试，比如贝尔实验室中的 Unix 后继项目，Plan9 等都在试图将系统全方位地以文件形式实现。

Linux 像任何一名生机勃勃的大学生一样决心走出这种困境：开发自己的操作系统。他开始写了一个简单的终端仿真程序，用于连接到本校的大型 Unix 系统上。他的终端仿真程序经过一学年的研发，不断改进和完善。不久，Linux 手上就有了虽不成熟但五脏俱全的 Unix。1991 年年底，他在 Internet 上发布了早期版本。

从此 Linux 便起航了，最初的 Linux 发布很快赢得了众多用户。而实际上，它成功的重要因素是，Linux 很快吸引了很多开发者、黑客对其代码进行修改和完善。由于其许可证条款的约定，Linux 迅速成为多人的合作开发项目。

到现在，Linux 早已羽翼丰满，它被广泛移植到 Alpha、ARM、PowerPC、SPARC、x86-64 等许多其他体系结构之上。如今 Linux 既被安装在最轻小的消费电子设备上，比如手表，同时也在服务规模最庞大的服务数据中心上，如超级计算机集群。今天，Linux 的商业前景也越来越被看好，不管是新成立的 Linux 专业公司 Red Hat 还是闻名遐迩的计算巨头 IBM，都提供林林总总的解决方案，从嵌入式系统、桌面环境一直到服务器。

Linux 是类 Unix 系统，但它不是 Unix。需要说明的是，尽管 Linux 借鉴了 Unix 的许多设计并且实现了 Unix 的 API（由 Posix 标准和其他 Single Unix Specification 定义的），但 Linux 没有像其他 Unix 变种那样直接使用 Unix 的源代码。必要的时候，它的实现可能和其他各种 Unix 的实现大相径庭，但它没有抛弃 Unix 的设计目标并且保证了应用程序编程接口的一致。

Linux 是一个非商业化的产品，这是它最让人感兴趣的特征。实际上 Linux 是一个互联网上的协作开发项目。尽管 Linux 被认为是 Linux 之父，并且现在依然是一个内核维护者，但开发工作其实是由一个结构松散的工作组协力完成的。事实上，任何人都可以开发内核。和该系统的大部分一样，Linux 内核也是自由（公开）软件^①。当然，也不是无限自由的。它使用 GNU 的 General Public License（GPL）第 2 版作为限制条款。这样做的结果是，你可以自由地获取内核代码并随意修改它，但如果你希望发布你修改过的内核，你也得保证让得到你的内核的人同时享有你曾经享受过的所有权利，当然，包括全部的源代码^②。

Linux 用途广泛，包含的东西也名目繁多。Linux 系统的基础是内核、C 库、工具集和系统的基本工具，如登录程序和 Shell。Linux 系统也支持现代的 X Windows 系统，这样就可以使用完整的图形用户桌面环境，如 GNOME。可以在 Linux 上使用的商业和自由软件数以千计。在这本书以后的部分，当我使用 Linux 这个词时，我其实说的是 Linux 内核。在容易引起混淆的地方，我会具体说明到底我想说的是整个系统还是内核。一般情况下，Linux 这个词汇主要还是指内核。

1.3 操作系统和内核简介

由于一些现行商业操作系统日趋庞杂及其设计上的缺陷，操作系统的精确定义并没有一个统一的标准。许多用户把他们在显示器屏幕上看到的东西理所当然地认为就是操作系统。通常，当

① 谁有兴趣了解 free VS open 的论战，请参见 <http://www.fsf.org> and <http://www.opensource.org>。

② 如果你没有读过 GNU GPL 2.0，你最好还是先读读它吧。内核代码树种的 .COPYING 文件就是它的一份拷贝。你也可以在 <http://www.fsf.org> 找到它。注意最新的 GNU GPL 已经是 3.0 版本了，但内核开发者们仍然决定继续使用 2.0 版本。

然在本书中也这么认为，操作系统是指在整个系统中负责完成最基本功能和系统管理的那些部分。这些部分应该包括内核、设备驱动程序、启动引导程序、命令行 Shell 或者其他种类的用户界面、基本的文件管理工具和系统工具。这些都是必不可少的东西——别以为只要有浏览器和播放器就行了。系统这个词其实包含了操作系统和所有运行在它之上的应用程序。

当然，本书的主题是内核。用户界面是操作系统的外在表象，内核才是操作系统的内在核心。系统其他部分必须依靠内核这部分软件提供的服务，像管理硬件设备、分配系统资源等。内核有时候被称作是**管理者**或者是**操作系统核心**。通常一个内核由负责响应中断的中断服务程序，负责管理多个进程从而分享处理器时间的调度程序，负责管理进程地址空间的内存管理程序和**网络、进程间通信**等系统服务程序共同组成。对于提供保护机制的现代系统来说，内核独立于普通应用程序，它一般处于系统态，拥有受保护的内存空间和访问硬件设备的所有权限。这种系统态和被保护起来的内存空间，统称为**内核空间**。相对的，应用程序在用户空间执行。它们只能看到允许它们使用的部分系统资源，并且只使用某些特定的系统功能，不能直接访问硬件，也不能访问内核划给别人的内存范围，还有其他一些使用限制。当内核运行的时候，系统以内核态进入内核空间执行。而执行一个普通用户程序时，系统将以用户态进入以用户空间执行。

在系统中运行的应用程序通过**系统调用**来与内核通信（见图 1-1）。应用程序通常调用库函数（比如 C 库函数）再由库函数通过系统调用界面，让内核代其完成各种不同任务。一些库调用提供了系统调用不具备的许多功能，在那些较为复杂的函数中，调用内核的操作通常只是整个工作的一个步骤而已。举个例子，拿 `printf()` 函数来说，它提供了数据的缓存和格式化等操作，而调用 `write()` 函数将数据写到控制台上只不过是其中的一个动作罢了。不过，也有一些库函数和系统调用就是一一对应的关系，比如，`open()` 库函数除了调用 `open()` 系统调用之外，几乎什么也不做。还有一些 C 库函数，像 `strcpy()`，根本就不需要直接调用系统级的操作。当一个应用程序执行一条系统调用，我们说内核正在代其执行。如果进一步解释，在这种情况下，应用程序被称为通过系统调用在内核空间运行，而内核被称为运行于进程上下文中。这种交互关系——应用程序通过系统调用界面陷入内核——是应用程序完成其工作的基本行为方式。

内核还要负责管理系统的硬件设备。现有的几乎所有的体系结构，包括全部 Linux 支持的体系结构，都提供了中断机制。当硬件设备想和系统通信的时候，它首先要发出一个异步的中断信号去打断处理器的执行，继而打断内核的执行。中断通常对应着一个中断号，内核通过这个中断号查找相应的**中断服务程序**，并调用这个程序响应和处理中断。举个例子，当你敲击键盘的时候，键盘控制器发送一个中断信号告知系统，键盘缓冲区有数据到来。内核注意到这个中断对应的中断号，调用相应的中断服务程序。该服务程序处理键盘数据然后通知键盘控制器可以继续输入数据了。为了保证同步，内核可以停用中止——既可以停止所有的中断也可以有选择地停止某个中断号对应的中断。许多操作系统的中断服务程序，包括 Linux 的，都不在进程上下文中执行。它们在一个与所有进程都无关的、专门的中断上下文中运行。之所以存在这样一个专门的执行环境，就是为了保证中断服务程序能够在第一时间响应和处理中断请求，然后快速地退出。

这些上下文代表着内核活动的范围。实际上我们可以将每个处理器在任何时间点上的活

动必然概括为下列三者之一：

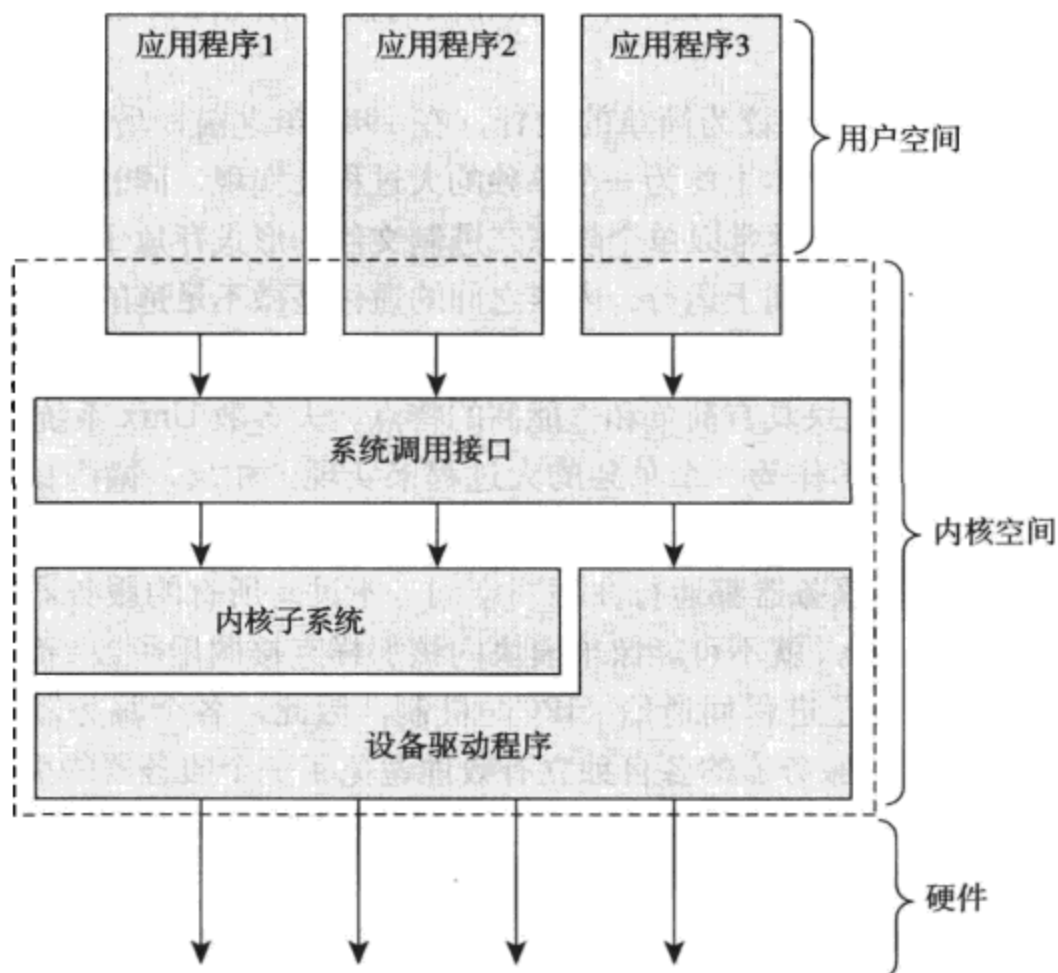


图 1-1 应用程序、内核和硬件的关系

- 运行于用户空间，执行用户进程。
- 运行于内核空间，处于进程上下文，代表某个特定的进程执行。
- 运行于内核空间，处于中断上下文，与任何进程无关，处理某个特定的中断。

以上所列几乎包括所有情况，即使边边角角的情况也不例外，例如，当 CPU 空闲时，内核就运行一个空进程，处于进程上下文，但运行于内核空间。

1.4 Linux 内核和传统 Unix 内核的比较

由于所有的 Unix 内核都同宗同源，并且提供相同的 API，现代的 Unix 内核存在许多设计上的相似之处（请看参考目录中我所推荐的关于传统 Unix 内核设计的相关书籍）。Unix 内核几乎毫无例外的都是一个不可分割的静态可执行库。也就是说，它们必须以巨大、单独的可执行块的形式在一个单独的地址空间中运行。Unix 内核通常需要硬件系统提供页机制（MMU）以管理内存。这种页机制可以加强对内存空间的保护，并保证每个进程都可以运行于不同的虚地址空间上。初期的 Linux 系统也需要 MMU 支持，但有一些特殊版本并不依赖于此。这无疑是一个简洁的设计，因为它可以使 Linux 系统运行在没有 MMU 的小型嵌入系统上。不过现实之中，即便很简单的嵌入系统都开始具备内存管理单元这种高级功能了。本书中，我们将重点关注支持 MMU 的 Linux 系统。

单内核与微内核设计之比较

操作系统内核可以分为两大阵营：单内核和微内核（第三阵营是外内核，主要用在科研系统中）。

单内核是两大阵营中一种较为简单的设计，在1980年之前，所有的内核都设计成单内核。所谓单内核就是把它从整体上作为一个单独的大过程来实现，同时也运行在一个单独的地址空间上。因此，这样的内核通常以单个静态二进制文件的形式存放于磁盘中。所有内核服务都在这样的一个大内核地址空间上运行。内核之间的通信是微不足道的，因为大家都运行在内核态，并身处同一地址空间：内核可以直接调用函数，这与用户空间应用程序没有什么区别。这种模式的支持者认为单模块具有简单和性能高的特点。大多数 Unix 系统都设计为单模块。

另一方面，微内核并不作为一个单独的大过程来实现。相反，微内核的功能被划分为多个独立的过程，每个过程叫做一个服务器。理想情况下，只有强烈请求特权服务的服务器才运行在特权模式下，其他服务器都运行在用户空间。不过，所有的服务器都保持独立并运行在各自的地址空间上。因此，就不可能像单模块内核那样直接调用函数，而是通过消息传递处理微内核通信：系统采用了进程间通信（IPC）机制，因此，各个服务器之间通过 IPC 机制互通消息，互换“服务”。服务器的各自独立有效地避免了一个服务器的失效祸及另一个。同样，模块化的系统允许一个服务器为了另一个服务器而换出。

因为 IPC 机制的开销多于函数调用，又因为会涉及内核空间与用户空间的上下文切换，因此，消息传递需要一定的周期，而单内核中简单的函数调用没有这些开销。结果，所有实际应用的基于微内核的系统都让大部分或全部服务器位于内核，这样，就可以直接调用函数，消除频繁的上下文切换。Windows NT 内核（Windows XP、Windows Vista 和 Windows 7 等基于此）和 Mach（Mac OS X 的组成部分）是微内核的典型实例。不管是 Windows NT 还是 Mac OS X，都在其新近版本中不让任何微内核服务器运行在用户空间，这违背了微内核设计的初衷。

Linux 是一个单内核，也就是说，Linux 内核运行在单独的内核地址空间上。不过，Linux 汲取了微内核的精华：其引以为豪的是模块化设计、抢占式内核、支持内核线程以及动态装载内核模块的能力。不仅如此，Linux 还避其微内核设计上性能损失的缺陷，让所有事情都运行在内核态，直接调用函数，无须消息传递。至今，Linux 是模块化的、多线程的以及内核本身可调度的操作系统，实用主义再次占了上风。

当 Linus 和其他内核开发者设计 Linux 内核时，他们并没有完全彻底地与 Unix 诀别。他们充分地认识到，不能忽视 Unix 的底蕴（特别是 Unix 的 API）。而由于 Linux 并没有基于某种特定的 Unix，Linus 和他的伙伴们对每个特定的问题都可以选择已知最理想的解决方案——在有些时候，当然也可以创造一些新的方案。Linux 内核与传统的 Unix 系统之间存在一些显著的差异：

- Linux 支持动态加载内核模块。尽管 Linux 内核也是单内核，可是允许在需要的时候动态地卸除和加载部分内核代码。
- Linux 支持对称多处理（SMP）机制，尽管许多 Unix 的变体也支持 SMP，但传统的 Unix 并不支持这种机制。

- Linux 内核可以抢占 (preemptive)。与传统的 Unix 变体不同, Linux 内核具有允许在内核运行的任务优先执行的能力。在其他各种 Unix 产品中, 只有 Solaris 和 IRIX 支持抢占, 但是大多数 Unix 内核不支持抢占。
- Linux 对线程支持的实现比较有意思: 内核并不区分线程和其他的一般进程。对于内核来说, 所有的进程都一样——只不过是其中的一些共享资源而已。
- Linux 提供具有设备类的面向对象的设备模型、热插拔事件, 以及用户空间的设备文件系统 (sysfs)。
- Linux 忽略了一些被认为是设计得很拙劣的 Unix 特性, 像 STREAMS, 它还忽略了那些难以实现的过时标准。
- Linux 体现了自由这个词的精髓。现有的 Linux 特性集就是 Linux 公开开发模型自由发展的结果。如果一个特性没有任何价值或者创意很差, 没有任何人会被迫去实现它。相反的, 针对变革, Linux 已经形成了一种值得称赞的态度: 任何改变都必须要通过简洁的设计及正确可靠的实现来解决现实中确实存在的问题。于是, 许多出现在某些 Unix 变种系统中, 那些出于市场宣传目的或没有普遍意义的一些特性, 如内核换页机制等都被毫不迟疑地摒弃了。

不管 Linux 和 Unix 有多大的不同, 它身上都深深地打上了 Unix 烙印。

1.5 Linux 内核版本

Linux 内核有两种: 稳定的和处于开发中的。稳定的内核具有工业级的强度, 可以广泛地应用和部署。新推出的稳定内核大部分都只是修正了一些 Bug 或是加入了一些新的设备驱动程序。另一方面处于开发中的内核中许多东西变化得都很快。而且由于开发者不断试验新的解决方案, 内核常常发生剧烈的变化。

Linux 通过一个简单的命名机制来区分稳定的和处于开发中的内核 (见图 1-2)。这种机制使用三个或者四个用 “.” 分隔的数字来代表不同内核版本。第一个数字是主版本号, 第二个数字是从版本号, 第三个数字是修订版本号, 第四个可选的数字为稳定版本号 (stable version)。从副版本号可以反映出该内核是一个稳定版本还是一个处于开发中的版本: 该数字如果是偶数, 那么此内核就是稳定版; 如果是奇数, 那么它就是开发版。举例来说, 版本号为 2.6.30.1 的内核, 它就是一个稳定版。这个内核的主版本号是 2, 从版本号是 6, 修订版本号是 30, 稳定版本号是 1。头两个数字在一起描述了 “内核系列” —— 在这个例子中, 就是 2.6 版内核系列。

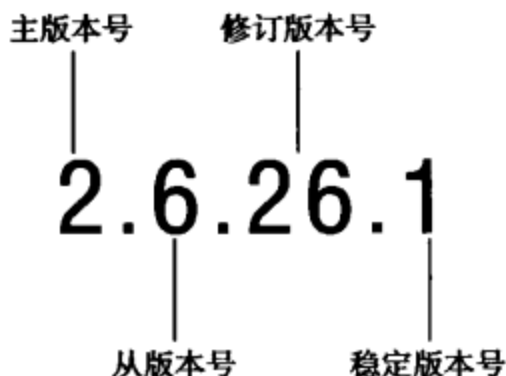


图 1-2 Kernel 版本命名规则

处于开发中的内核一般要经历几个阶段。最开始，内核开发者们开始试验新的特性，这时候出现错误和混乱是在所难免的。经过一段时间，系统渐渐成熟，最终会有一个特性审定的声明。这时候，Linus 就不再接受新的特性了，而对已有特性所进行的后续工作会继续进行。当 Linus 认为这个新内核确实是趋于稳定后，就开始审定代码。这以后，就只允许再向其中加入修改 bug 的代码了。在经过一个短暂（希望如此）的准备期之后，Linus 会将这个内核作为一个新的稳定版推出。例如，1.3 系列的开发版稳定在 2.0，而 2.5 稳定在 2.6。

在一个特定的系列下，Linus 会定期发布新内核。每个新内核都是一个新的修订版本。比如 2.6 内核系列的第一个版本是 2.6.0，第二个版本是 2.6.1。这些修订版包含了 BUG 修复、新的驱动和一些新特性。但是，像 2.6.3 和 2.6.4 修订版本之间的差异是很微小的。

这种开发方式一直持续到 2004 年，当时在受邀参加的 Linux 开发者峰会上，内核开发者们确定延长 2.6 内核系列，从而推迟进入到 2.7 系列的步伐。原因是 2.6 版本的内核已经被广泛接受、其已经证明了稳定成熟，而那些还不成熟的新特性其实并非人们所需。如今看来 2.6 版本内核的稳定出色无疑证明了该方针是多么英明。在编写本书时，2.7 版本内核仍未提上议程，而且也看不出任何启动迹象。相反，每个 2.6 系列内核的修订版本发布变得越发长久，每个修订版都伴随有一个最小的开发版系列（称其微缩开发版）。Andrew Morton，Linus 的副手，重新定义了他所维护的 2.6-mm 代码树（它曾经用于内存管理相关改动的测试版本），使其成为一个通用目的测试版本。任何尚未稳定的修改都将首先进入 2.6-mm 树中，等其稳定后，再进入某个 2.6 的微缩开发版。如此策略的结果是：最近几年，每一个 2.6 系列的修订版本（比如 2.6.29）都会较其前身有深刻的变化，也都会经历数月才面世。这种“微缩开发版方式”被证明是可行的、成功的，它更有利于在引入新特性的同时，维持系统的稳定性。想必在近期内开发策略不会改弦易辙，事实上内核开发者们就新版本的发布流程延续目前方式已经达成了一致意见。

为了解决版本发布周期变长的副作用，内核开发者们引入了上面提到的稳定版本号。这个稳定版本号（如 2.6.32.8 中的 8）包含了关键性 bug 的修改，并且常会向前移植处于开发版内核（如 2.6.33）的重要修改。依靠这种方式，以前版本保证了仍然能将重点放在稳定性上。

1.6 Linux 内核开发者社区

当你开始开发内核代码时，你就成为全球内核开发社区的一分子了。这个社区最重要的论坛是 linux kernel mailing list（常缩写为 lkml）。你可以在 <http://vegr.kernel.org> 上订阅邮件。要注意的是这个邮件列表流量很大，每天有超过几百条的消息，所以其他的订阅者（包括所有的核心开发人员，甚至包括 Linus 本人）可没有心思听人说废话。这个邮件列表可以给从事内核开发的人提供价值无穷的帮助，在这里，你可以寻找测试人员，接受评论（peer review），向人求助。

后续内容列出了内核开发过程的全景，并详尽地描述了如何成功地加入到内核开发社区中去。但是要明白，在 Linux 内核邮件列表中潜伏（安静地阅读）是你阅读本书的最好补充。

1.7 小结

这是一本关于 Linux 内核的书：内核的目标，为达到目标所进行的设计以及设计的实现。这本书侧重实用，同时在讲述工作原理时会结合理论联系实践。我的目标是让你从一个业内人士的

视角来欣赏和理解 Linux 内核的设计和实现之美。力求以一种有趣的方式（伴随着我个人在开发内核过程中收集的种种奇闻逸事和方法技巧）引导你走过跌跌撞撞的起步阶段。无论你是立志于开发内核代码，或者进行驱动开发，甚至只是希望能更好地了解 Linux 操作系统，你都将 从本书受益。

当你阅读本书时，我希望你有一台装有 Linux 的机器，我希望你能够看到内核代码。其实，这很理想了，因为这意味着你是一位 Linux 的使用者，并且早已经开始拿起手术刀对着源代码进行探索了，只不过需要一份结构图以便对整个经脉有个总体把握罢了。相反，你可能没有使用过 Linux，只是在好奇心的驱使下希望了解一些内核设计的秘密而已。但是，如果你的目的只是撰写自己的代码，那么，源代码的作用无可替代。而且，你不需要付出任何代价，尽管用吧。

好了，最重要的是，在其中寻找快乐吧。



第 ② 章

从内核出发

在这一章，我们将介绍 Linux 内核的一些基本常识：从何处获取源码，如何编译它，又如何安装新内核。那么，让我们考察一下内核程序与用户空间程序的差异，以及内核中所使用的通用编程结构。虽然内核在很多方面有其独特性，但从现在来看，它和其他大型软件项目并无多大差别。

2.1 获取内核源码

登录 Linux 内核官方网站 <http://www.kernel.org>，可以随时获取当前版本的 Linux 源代码，可以是完整的压缩形式（使用 tar 命令创建的一个压缩文件），也可以是增量补丁形式。

除特殊情况下需要 Linux 源码的旧版本外，一般都希望拥有最新的代码。kernel.org 是源码的库存之处，那些领导潮流的内核开发者所发布的增量补丁也放在这里。

2.1.1 使用 Git

在过去的几年中，Linus 和他领导的内核开发者们开始使用一个新版本的控制系统来管理 Linux 内核源代码。Linus 创造的这个系统称为 Git。与 CSV 这样的传统的版本控制系统不同，Git 是分布式的，它的用法和 workflows 对许多开发者来说都很陌生。我强烈建议使用 Git 来下载和管理 Linux 内核源代码。

你可以使用 Git 来获取最新提交到 Linus 版本树的一个副本：

```
$ git clone git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux-2.6.git
```

当下载代码后，你可以更新你的分支到 Linus 的最新分支：

```
$ git pull
```

有了这两个命令，就可以获取并随时保持与内核官方的代码树一致。要提交和管理自己的修改，请看第 20 章。关于 Git 的全面讨论已经超出了本书的范围，许多在线资源都提供了有效的指导。

2.1.2 安装内核源代码

内核压缩以 GNU zip (gzip) 和 bzip2 两种形式发布。bzip2 是默认和首选形式，因为它在压缩上比 gzip 更有优势。以 bzip2 形式发布的 Linux 内核叫做 linux-x.y.z.tar.bz2，这里 x.y.z 是内核源码的具体版本。下载了源代码之后，就可以轻而易举地对其解压。如果压缩形式是 bzip2，则

运行：

```
$ tar xvjf linux-x.y.z.tar.bz2
```

如果压缩形式是 GNU 的 zip，则运行：

```
$ tar xvzf linux-x.y.z.tar.gz
```

解压后的源代码位于 linux-x.y.z. 目录下。如果你是使用 git 获取和管理内核源代码，那么就不需要下载压缩文件，只要像前面描述的那样运行 git clone 命令，git 就会下载并且解压最新的源代码。

何处安装并触及源码

内核源码一般安装在 /usr/src/linux 目录下。但请注意，不要把这个源码树用于开发，因为编译你的 C 库所用的内核版本就链接到这棵树。此外，不要以 root 身份对内核进行修改，而应当是建立自己的主目录，仅以 root 身份安装新内核。即使在安装新内核时，/usr/src/linux 目录都应当原封不动。

2.1.3 使用补丁

在 Linux 内核社区中，补丁是通用语。你可以以补丁的形式发布对代码的修改，也可以以补丁的形式接收其他人所做的修改。增量补丁可以作为版本转移的桥梁。你不再需要下载庞大的内核源码的全部压缩，而只需给旧版本打上一个增量补丁，让其旧貌换新颜。这不仅节约了带宽，还省了时间。要应用增量补丁，从你的内部源码树开始，只需运行：

```
$ patch -p1 < ../patch-x.y.z
```

一般来说，一个给定版本的内核补丁总是打在前一个版本上。

有关创建和应用补丁更深入的讨论会在后续章节进行。

2.2 内核源码树

内核源码树由很多目录组成，而大多数目录又包含更多的子目录。源码树的根目录及其子目录如表 2-1 所示。

表 2-1 内核源码树的根目录描述

目 录	描 述
arch	特定体系结构的源码
block	块设备 I/O 层
crypto	加密 API
Documentation	内核源码文档
drivers	设备驱动程序
firmware	使用某些驱动程序而需要的设备固件
fs	VFS 和各种文件系统

(续)

目 录	描 述
include	内核头文件
init	内核引导和初始化
ipc	进程间通信代码
kernel	像调度程序这样的核心子系统
lib	通用内核函数
mm	内存管理子系统和 VM
net	网络子系统
samples	示例, 示范代码
scripts	编译内核所用的脚本
security	Linux 安全模块
sound	语音子系统
usr	早期用户空间代码 (所谓的 initramfs)
tools	在 Linux 开发中有用的工具
virt	虚拟化基础结构

在源码树根目录中的很多文件值得提及。COPYING 文件是内核许可证 (GNU GPL v2)。CREDITS 是开发了很多内核代码的开发者列表。MAINTAINERS 是维护者列表, 它们负责维护内核子系统和驱动程序。Makefile 是基本内核的 Makefile。

2.3 编译内核

编译内核易如反掌。让人叹为观止的是, 这实际上比编译和安装像 glibc 这样的系统级组件还要简单。2.6 内核提供了一套新工具, 使编译内核更加容易, 比早期发布的内核有了长足的进步。

2.3.1 配置内核

因为 Linux 源码随手可得, 那就意味着在编译它之前可以配置和定制。的确, 你可以把自己需要的特定功能和驱动程序编译进内核。在编译内核之前, 首先你必须配置它。由于内核提供了数不胜数的功能, 支持了难以计数的硬件, 因而有许多东西需要配置。可以配置的各种选项, 以 CONFIG_FEATURE 形式表示, 其前缀为 CONFIG。例如, 对称多处理器 (SMP) 的配置选项为 CONFIG_SMP。如果设置了该选项, 则 SMP 启用, 否则, SMP 不起作用。配置选项既可以用来决定哪些文件编译进内核, 也可以通过预处理命令处理代码。

这些配置项要么是二选一, 要么是三选一。二选一就是 yes 或 no。比如 CONFIG_PREEMPT 就是二选一, 表示内核抢占功能是否开启。三选一可以是 yes、no 或 module。module 意味着该配置项被选定了, 但编译的时候这部分功能的实现代码是以模块 (一种可以动态安装的独立代码段) 的形式生成。在三选一的情况下, 显然 yes 选项表示把代码编译进主内核映像中, 而不是作为一个模块。驱动程序一般都用三选一的配置项。

配置选项也可以是字符串或整数。这些选项并不控制编译过程，而只是指定内核源码可以访问的值，一般以预处理宏的形式表示。比如，配置选项可以指定静态分配数组的大小。

销售商提供的内核，像 Canonical 的 Ubuntu 或者 Red Hat 的 Fedora，他们的发布版中包含了预编译的内核，这样的内核使得所需的功能得以充分地启用，并几乎把所有的驱动程序都编译成模块。这就为大多数硬件作为独立的模块提供了坚实的内核支持。但是，话又说回来，如果你是一个内核黑客，你应当编译自己的内核，并按自己的意愿决定包括或不包含哪一模块。

内核提供了各种不同的工具来简化内核配置。最简单的一种是一个字符界面下的命令行工具：

```
$ make config
```

该工具会逐一遍历所有配置项，要求用户选择 yes、no 或是 module（如果是三选一的话）。由于这个过程往往要耗费掉很长时间，所以，除非你的工作是按小时计费的，否则应该多利用基于 ncurses 库编制的图形界面工具：

```
$ make menuconfig
```

或者，是用基于 gtk+ 的图形工具：

```
$ make gconfig
```

这三种工具将所有配置项分门别类放置，比如按“处理器类型和特点”。你可以按类移动、浏览内核选项，当然也可以修改其值。

这条命令会基于默认的配置为你的体系结构创建一个配置：

```
$ make defconfig
```

尽管这些缺省值有点随意性（在 i386 上，据说那就是 Linus 的配置），但是，如果你从未配置过内核，那它们会提供一个良好的开端。赶快行动吧，运行这条命令，然后回头看看，确保为你的硬件所配置的选项是启用的。

这些配置项会被存放在内核代码树根目录下的 .config 文件中。你很容易就能找到它（内核开发者差不多都能找到），并且可以直接修改它。在这里面查找和修改内核选项也很容易。在你修改过配置文件之后，或者在用已有的配置文件配置新的代码树的时候，你应该验证和更新配置：

```
$ make oldconfig
```

事实上，在编译内核之前你都应该这么做。

配置选项 CONFIG_IKCONFIG_PROC 把完整的压缩过的内核配置文件存放在 /proc/config.gz 下，这样当你编译一个新内核的时候就可以方便地克隆当前的配置。如果你目前的内核已经启用了此选项，就可以从 /proc 下复制出配置文件并且使用它来编译一个新内核：

```
$ zcat /proc/config.gz > .config  
$ make oldconfig
```

一旦内核配置好了（不论你是如何配置的），就可以使用一个简单的命令来编译它了：

```
$ make
```

这跟 2.6 以前的版本不同，你不用在每次编译内核之间都运行 make dep 了——代码之间的

依赖关系会自动维护。你也无须再指定像老版本中 bzImage 这样的编译方式或独立地编译模块，默认的 Makefile 规则会打点这一切。

2.3.2 减少编译的垃圾信息

如果你想尽量少地看到垃圾信息，却又不希望错过错误报告与警告信息的话，你可以用以下命令来对输出进行重定向：

```
$ make > .. /detritus
```

一旦你需要查看编译的输出信息，你可以查看这个文件。不过，因为错误和警告都会会在屏幕上显示，所以你需要看这个文件的可能性不大。事实上，我只不过输入如下命令：

```
$ make > /dev/null
```

就可把无用的输出信息重定向到永无返回值的黑洞 /dev/null。

2.3.3 衍生多个编译作业

make 程序能把编译过程拆分成多个并行的作业。其中的每个作业独立并发地运行，这有助于极大地加快多处理器系统上的编译过程，也有利于改善处理器的利用率，因为编译大型源代码树也包括 I/O 等待所花费的时间（也就是处理器空下来等待 I/O 请求完成所花费的时间）。

默认情况下，make 只衍生一个作业，因为 Makefiles 常会出现不正确的依赖信息。对于不正确的依赖，多个作业可能会互相踩踏，导致编译过程出错。当然，内核的 Makefiles 没有这样的编码错误，因此衍生出的多个作业编译不会出现失败。为了以多个作业编译内核，使用以下命令：

```
$ make -jn
```

这里，n 是要衍生出的作业数。在实际中，每个处理器上一般衍生出一个或者两个作业。例如，在一个 16 核处理器上，你可以输入如下命令：

```
$ make -j32 > /dev/null
```

利用出色的 distcc 或者 ccache 工具，也可以动态地改善内核的编译时间。

2.3.4 安装新内核

在内核编译好之后，你还需要安装它。怎么安装就和体系结构以及启动引导工具（boot loader）息息相关了——查阅启动引导工具的说明，按照它的指导将内核映像拷贝到合适的位置，并且按照启动要求安装它。一定要保证随时有一个或两个可以启动的内核，以防新编译的内核出现问题。

例如，在使用 grub 的 x86 系统上，可能需要把 arch/i386/boot/bzImage 拷贝到 /boot 目录下，像 vmlinuz-version 这样命名它，并且编辑 /etc/grub/grub.conf 文件，为新内核建立一个新的启动项。使用 LILO 启动的系统应当编辑 /etc/lilo.conf，然后运行 lilo。

所幸，模块的安装是自动的，也是独立于体系结构的。以 root 身份，只要运行：

```
% make modules_install
```

就可以把所有已编译的模块安装到正确的主目录 `/lib/modules` 下。

编译时也会在内核代码树的根目录下创建一个 `System.map` 文件。这是一份符号对照表，用以将内核符号和它们的起始地址对应起来。调试的时候，如果需要把内存地址翻译成容易理解的函数名以及变量名，这就会很有用。

2.4 内核开发的特点

相对于用户空间内应用程序的开发，内核开发有一些独特之处。尽管这些差异并不会使开发内核代码的难度超过开发用户代码，但它们依然有很大不同。

这些特点使内核成了一只性格迥异的猛兽。一些常用的准则被颠覆了，而又必须建立许多全新的准则。尽管有许多差异一目了然（人人都知道内核可以做它想做的任何事），但还是有一些差异晦暗不明。最重要的差异包括以下几种：

- 内核编程时既不能访问 C 库也不能访问标准的 C 头文件。
- 内核编程时必须使用 GNU C。
- 内核编程时缺乏像用户空间那样的内存保护机制。
- 内核编程时难以执行浮点运算。
- 内核给每个进程只有一个很小的定长堆栈。
- 由于内核支持异步中断、抢占和 SMP，因此必须时刻注意同步和并发。
- 要考虑可移植性的重要性。

让我们仔细考察一下这些要点，所有内核开发者必须牢记以上要点。

2.4.1 无 libc 库抑或无标准头文件

与用户空间的应用程序不同，内核不能链接使用标准 C 函数库——或者其他的那些库也不行。造成这种情况的原因有许多，其中就包括先有鸡还是先有蛋这个悖论。不过最主要的原因还是速度和大小。对内核来说，完整的 C 库——哪怕是它的一个子集，都太大且太低效了。

别着急，大部分常用的 C 库函数在内核中都已经得到了实现。比如操作字符串的函数组就位于 `lib/string.c` 文件中。只要包含 `<linux/string.h>` 头文件，就可以使用它们。

头文件

当我在本书中谈及头文件时，都指的是组成内核源代码树的内核头文件。内核源代码文件不能包含外部头文件，就像它们不能用外部库一样。

基本的头文件位于内核源代码树顶级目录下的 `include` 目录中。例如，头文件 `<linux/inotify.h>` 对应内核源代码树的 `include/linux/inotify.h`。

体系结构相关的头文件集位于内核源代码树的 `arch/<architecture>/include/asm` 目录下。例如，如果编译的是 x86 体系结构，则体系结构相关的头文件就是 `arch/x86/include/asm`。内核代码通过以 `asm/` 为前缀的方式包含这些头文件，例如 `<asm/ioctl.h>`。

在所有没有实现的函数中，最著名的就数 `printf()` 函数了。内核代码虽然无法调用 `printf()`，但它提供的 `printk()` 函数几乎与 `printf()` 相同。`printk()` 函数负责把格式化好的字符串拷贝到内核日志缓冲区上，这样，`syslog` 程序就可以通过读取该缓冲区来获取内核信息。`printk()` 的用法很像 `printf()`：

```
printk("Hello world! A string: '%s' and an integer: '%d'\n", str, i);
```

`printk()` 和 `printf()` 之间的一个显著区别在于，`printk()` 允许你通过指定一个标志来设置优先级。`syslogd` 会根据这个优先级标志来决定在什么地方显示这条系统消息。下面是一个使用这种优先级标志的例子：

```
printk(KERN_ERR "this is an error!\n");
```

注意 在 `KERN_ERR` 和要打印的消息之间没有逗号，这样写是别有用意的。优先级标志是预处理程序定义的一个描述性字符串，在编译时优先级标志就与要打印的消息绑在一起处理。贯穿整本书，我们会使用 `printk()`。

2.4.2 GNU C

像所有自视清高的 Unix 内核一样，Linux 内核是用 C 语言编写的。让人略感惊讶的是，内核并不完全符合 ANSI C 标准。实际上，只要有可能，内核开发者总是要用到 `gcc` 提供的许多语言的扩展部分。（`gcc` 是多种 GNU 编译器的集合，它包含的 C 编译器既可以编译内核，也可以编译 Linux 系统上用 C 语言写的其他代码。）

内核开发者使用的 C 语言涵盖了 ISO C99^① 标准和 GNU C 扩展特性。这其中的种种变化把 Linux 内核推向了 `gcc` 的怀抱，尽管目前出现了一些新的编译器如 Intel C，已经支持了足够多的 `gcc` 扩展特性，完全可以用来编译 Linux 内核了。最早支持 `gcc` 的版本是 3.2，但是推荐使用 `gcc` 4.4 或之后的版本。Linux 内核用到的 ISO C99 标准的扩展没有什么特别之处，而且 C99 作为 C 语言官方标准的修订本，不可能有大的或是激进的变化。让人感兴趣的，与标准 C 语言有区别的，通常也是人们不熟悉的那些变化，多数集中在 GNU C 上。就让我们研究一下内核代码中所使用到的 C 语言扩展中让人感兴趣的那部分吧，这些变化使内核代码有别于你所熟悉的其他项目。

1. 内联 (inline) 函数

C99 和 GNU C 均支持内联函数。`inline` 这个名称^②就可以反映出它的工作方式，函数会在它所调用的位置上展开。这么做可以消除函数调用和返回所带来的开销（寄存器存储和恢复）。而且，由于编译器会把调用函数的代码和函数本身放在一起进行优化，所以也有进一步优化代码的可能。不过，这么做是有代价的（天下没有免费的午餐），代码会变长，这也就意味着占用更多

① ISO C99 是 ISO C 的最新修订版。C99 相对于前一个修订版 C90 做了许多加强，ISO C99 引入了指定初始化，可变长度的数组，C++ 风格的注释，`long long` 和 `complex` 数据类型，但是 linux 内核只使用了 C99 特性的一个子集。

② 译者注：`inline` 翻译成内联似乎并不贴切，直译应该是“在字里行间展开”的意思，不过约定俗成，我们也把它翻译成“内联”。

的内存空间或者占用更多的指令缓存。内核开发者通常把那些对时间要求比较高，而本身长度又比较短的函数定义成内联函数。如果一个函数较大，会被反复调用，且没有特别的时间上的限制，我们并不赞成把它做成内联函数。

定义一个内联函数的时候，需要使用 `static` 作为关键字，并且用 `inline` 限定它。比如：

```
static inline void wolf(unsigned long tail_size)
```

内联函数必须在使用之前就定义好，否则编译器就没法把这个函数展开。实践中一般在头文件中定义内联函数。由于使用了 `static` 作为关键字进行限制，所以编译时不会为内联函数单独建立一个函数体。如果一个内联函数仅仅在某个源文件中使用，那么也可以把它定义在该文件开始的地方。

在内核中，为了类型安全和易读性，优先使用内联函数而不是复杂的宏。

2. 内联汇编

`gcc` 编译器支持在 C 函数中嵌入汇编指令。当然，在内核编程的时候，只有知道对应的体系结构，才能使用这个功能。

我们通常使用 `asm()` 指令嵌入汇编代码。例如，下面这条内联汇编指令用于执行 x86 处理器的 `rdtsc` 指令，返回时间戳（tsc）寄存器的值：

```
unsigned int low, high;
asm volatile("rdtsc" : "=a" (low), "=d" (high));
/* low 和 high 分别包含 64 位时间戳的低 32 位和高 32 位 */
```

Linux 的内核混合使用了 C 语言和汇编语言。在逼近体系结构的底层或对执行时间要求严格的地方，一般使用的是汇编语言。而内核其他部分的大部分代码是用 C 语言编写的。

3. 分支声明

对于条件选择语句，`gcc` 内建了一条指令用于优化，在一个条件经常出现，或者该条件很少出现的时候，编译器可以根据这条指令对条件分支选择进行优化。内核把这条指令封装成了宏，比如 `likely()` 和 `unlikely()`，这样使用起来比较方便。

例如，下面是一个条件选择语句：

```
if (error) {
    /* ... */
}
```

如果想要把这个选择标记成绝少发生的分支：

```
/* 我们认为 error 绝大多数时间都会为 0...*/
if (unlikely(error)) {
    /* ... */
}
```

相反，如果我们想把一个分支标记为通常为真的选择：

```
/* 我们认为 success 通常都不会为 0 */
if (likely(success)) {
    /* ... */
}
```

在你想要对某个条件选择语句进行优化之前，一定要搞清楚其中是不是存在这么一个条件，在绝大多数情况下都会成立。这点十分重要：如果你的判断正确，确实是这个条件占压倒性的地位，那么性能会得到提升；如果你搞错了，性能反而会下降。正如上面这些例子所示，通常在对一些错误条件进行判断的时候会用到 `unlikely()` 和 `likely()`。你可以猜到，`unlikely()` 在内核中会得到更广泛的使用，因为 `if` 语句往往判断一种特殊情况。

2.4.3 没有内存保护机制

如果一个用户程序试图进行一次非法的内存访问，内核就会发现这个错误，发送 `SIGSEGV` 信号，并结束整个进程。然而，如果是内核自己非法访问了内存，那后果就很难控制了。（毕竟，有谁能照顾内核呢？）内核中发生的内存错误会导致 `oops`，这是内核中出现的最常见的一类错误。在内核中，不应该去做访问非法的内存地址，引用空指针之类的事情，否则它可能会死掉，却根本不告诉你一声——在内核里，风险常常会比外面大一些。

此外，内核中的内存都不分页。也就是说，你每用掉一个字节，物理内存就减少一个字节。所以，在你想往内核里加入什么新功能的时候，要记住这一点。

2.4.4 不要轻易在内核中使用浮点数

在用户空间的进程内进行浮点操作的时候，内核会完成从整数操作到浮点数操作的模式转换。在执行浮点指令时到底会做些什么，因体系结构不同，内核的选择也不同，但是，内核通常捕获陷阱并着手于整数到浮点方式的转变。

与用户空间进程不同，内核并不能完美地支持浮点操作，因为它本身不能陷入。在内核中使用浮点数时，除了要人工保存和恢复浮点寄存器，还有其他一些琐碎的事情要做。如果要直截了当地回答，那就是：别这么做了，除了一些极少的情况，不要在内核中使用浮点操作。

2.4.5 容积小而固定的栈

用户空间的程序可以从栈上分配大量的空间来存放变量，甚至巨大的结构体或者是包含数以千计的数据项的数组都没有问题。之所以可以这么做，是因为用户空间的栈本身比较大，而且还能动态地增长（年长的开发者回想一下 DOS 那个年代，这种低级的操作系统即使在用户空间也只有固定大小的栈）。

内核栈的准确大小随体系结构而变。在 x86 上，栈的大小在编译时配置，可以是 4KB 也可以是 8KB。从历史上说，内核栈的大小是两页，这就意味着，32 位机的内核栈是 8KB，而 64 位机是 16KB，这是固定不变的。每个处理器都有自己的栈。

关于内核栈的更多内容，会在后面的章节中讨论。

2.4.6 同步和并发

内核很容易产生竞争条件。和单线程的用户空间程序不同，内核的许多特性都要求能够并发地访问共享数据，这就要求有同步机制以保证不出现竞争条件，特别是：

- Linux 是抢占多任务操作系统。内核的进程调度程序即兴对进程进行调度和重新调度。内核必须和这些任务同步。
- Linux 内核支持对称多处理器系统 (SMP)。所以, 如果没有适当的保护, 同时在两个或两个以上的处理器上执行的内核代码很可能会同时访问共享的同一个资源。
- 中断是异步到来的, 完全不顾及当前正在执行的代码。也就是说, 如果不加以适当的保护, 中断完全有可能在代码访问资源的时候到来, 这样, 中段处理程序就有可能访问同一资源。
- Linux 内核可以抢占。所以, 如果不加以适当的保护, 内核中一段正在执行的代码可能会被另外一段代码抢占, 从而有可能导致几段代码同时访问相同的资源。

常用的解决竞争的办法是自旋锁和信号量。我们将在后面的章节中详细讨论同步和并发执行。

2.4.7 可移植性的重要性

尽管用户空间的应用程序不太注意移植问题, 然而 Linux 却是一个可移植的操作系统, 并且要一直保持这种特点。也就是说, 大部分 C 代码应该与体系结构无关, 在许多不同体系结构的计算机上都能够编译和执行, 因此, 必须把与体系结构相关的代码从内核代码树的特定目录中适当地分离出来。

诸如保持字节序、64 位对齐、不假定字长和页面长度等一系列准则都有助于移植性。对移植性的深度讨论将在后面的章节中进行。

2.5 小结

毫无疑问, 内核有独一无二的特质。它实施自己的规则和奖罚措施, 拥有整个系统的最高管理权。当然, Linux 内核的复杂性和高门槛与其他大型软件项目并无差异。在内核开发之路上最重要的步骤是要意识到内核并没有那么可怕。陌生是肯定的, 但真的就不可逾越? 事实并非如此。

本章和以前的章节为贯穿本书剩余章节所讨论的主题奠定了基础。在后续的每一章中, 我们都会涵盖内核的一个具体概念或子系统。在探索的征途中, 最重要的是要阅读和修改内核源代码, 只有通过实际的阅读和实践才会理解内核。内核源代码是可以免费获取的, 直接用就可以了!

第③章

进程管理

本章引入进程的概念，进程是 Unix 操作系统抽象概念中最基本的一种。其中涉及进程的定义以及相关的概念，比如线程；然后讨论 Linux 内核如何管理每个进程：它们在内核中如何被列举，如何创建，最终又如何消亡。我们拥有操作系统就是为了运行用户程序，因此，进程管理就是所有操作系统的核心所在，Linux 也不例外。

3.1 进程

进程就是处于执行期的程序（目标码存放在某种存储介质上）。但进程并不仅仅局限于一段可执行程序代码（Unix 称其为代码段，text section）。通常进程还要包含其他资源，像打开的文件，挂起的信号，内核内部数据，处理器状态，一个或多个具有内存映射的内存地址空间及一个或多个执行线程（thread of execution），当然还包括用来存放全局变量的数据段等。实际上，进程就是正在执行的程序代码的实时结果。内核需要有效而又透明地管理所有细节。

执行线程，简称线程（thread），是在进程中活动的对象。每个线程都拥有一个独立的程序计数器、进程栈和一组进程寄存器。内核调度的对象是线程，而不是进程。在传统的 Unix 系统中，一个进程只包含一个线程，但现在的系统中，包含多个线程的多线程程序司空见惯。稍后你会看到，Linux 系统的线程实现非常特别：它对线程和进程并不特别区分。对 Linux 而言，线程只不过是一种特殊的进程罢了。

在现代操作系统中，进程提供两种虚拟机制：虚拟处理器和虚拟内存。虽然实际上可能是许多进程正在分享一个处理器，但虚拟处理器给进程一种假象，让这些进程觉得自己在独享处理器。第 4 章将详细描述这种虚拟机制。而虚拟内存让进程在分配和管理内存时觉得自己拥有整个系统的所有内存资源。第 12 章将描述虚拟内存机制。有趣的是，注意在线程之间[⊖]可以共享虚拟内存，但每个都拥有各自的虚拟处理器。

程序本身并不是进程，进程是处于执行期的程序以及相关的资源的总称。实际上，完全可能存在两个或多个不同的进程执行的是同一个程序。并且两个或两个以上并存的进程还可以共享许多诸如打开的文件、地址空间之类的资源。

无疑，进程在创建它的时刻开始存活。在 Linux 系统中，这通常是调用 fork() 系统的结果，该系统调用通过复制一个现有进程来创建一个全新的进程。调用 fork() 的进程称为父进程，新产生的进程称为子进程。在该调用结束时，在返回点这个相同位置上，父进程恢复执行，子进程开始执行。fork() 系统调用从内核返回两次：一次回到父进程，另一次回到新产生的子进程。

⊖ 这里是指包含在同一个进程中的线程。——译者注

通常，创建新的进程都是为了立即执行新的、不同的程序，而接着调用 `exec()` 这组函数就可以创建新的地址空间，并把新的程序载入其中。在现代 Linux 内核中，`fork()` 实际上是由 `clone()` 系统调用实现的，后者将在后面讨论。

最终，程序通过 `exit()` 系统调用退出执行。这个函数会终结进程并将其占用的资源释放掉。父进程可以通过 `wait4()`^① 系统调用查询子进程是否终结，这其实使得进程拥有了等待特定进程执行完毕的能力。进程退出执行后被设置为僵死状态，直到它的父进程调用 `wait()` 或 `waitpid()` 为止。

注意 进程的另一个名字是任务 (task)。Linux 内核通常把进程也叫做任务。本书会交替使用这两个术语，不过我所说的任务通常指的是从内核观点所看到的进程。

3.2 进程描述符及任务结构

内核把进程的列表存放在叫做任务队列 (task list^②) 的双向循环链表中。链表中的每一项都是类型为 `task_struct`、称为进程描述符 (process descriptor) 的结构，该结构定义在 `<linux/sched.h>` 文件中。进程描述符中包含一个具体进程的所有信息。

`task_struct` 相对较大，在 32 位机器上，它大约有 1.7KB。但如果考虑到该结构内包含了内核管理一个进程所需的所有信息，那么它的大小也算相当小了。进程描述符中包含的数据能完整地描述一个正在执行的程序：它打开的文件，进程的地址空间，挂起的信号，进程的状态，还有其他更多信息 (见图 3-1)。

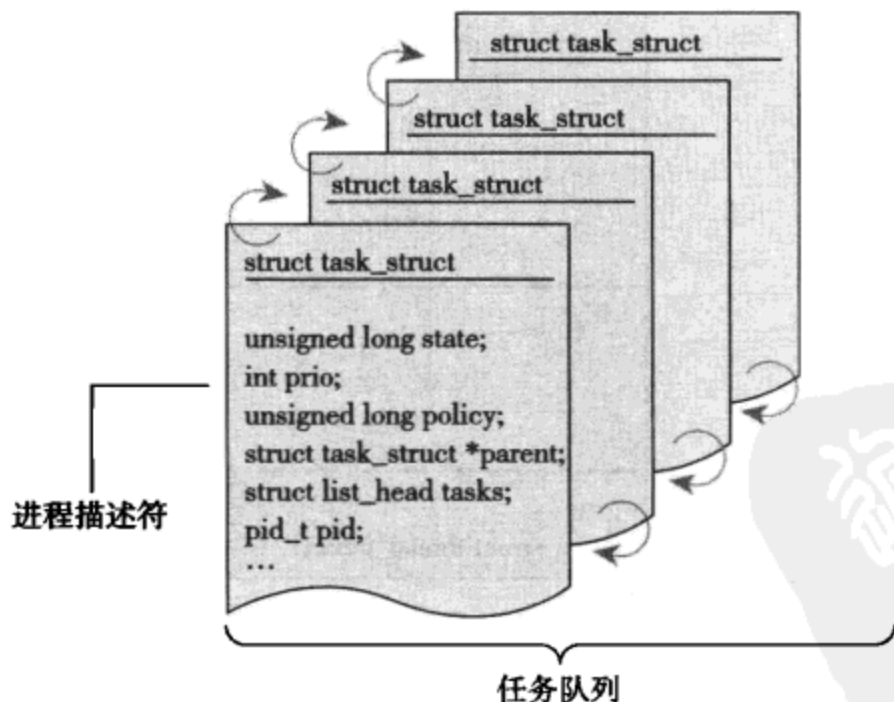


图 3-1 进程描述符及任务队列

- ① 由内核负责实现 `wait4()` 系统调用。Linux 系统通过 C 库通常要提供 `wait()`、`waitpid()`、`wait3()` 和 `wait4()` 函数。虽然有些细微的语意差别，但所有函数都返回关于终止进程的状态。
- ② 有些介绍操作系统的教材称这为任务数组 (task array)。由于 Linux 实现时使用的是队列而不是静态数组，所以就称作任务队列。

3.2.1 分配进程描述符

Linux 通过 slab 分配器分配 `task_struct` 结构，这样能达到对象复用和缓存着色 (cache coloring) (参见第 12 章) 的目的^①。在 2.6 以前的内核中，各个进程的 `task_struct` 存放在它们内核栈的尾端。这样做是为了让那些像 x86 那样寄存器较少的硬件体系结构只要通过栈指针就能计算出它的位置，而避免使用额外的寄存器专门记录。由于现在用 slab 分配器动态生成 `task_struct`，所以只需在栈底 (对于向下增长的栈来说) 或栈顶 (对于向上增长的栈来说) 创建一个新的结构 `struct thread_info`^② (见图 3-2)。

在 x86 上，`struct thread_info` 在文件 `<asm/thread_info.h>` 中定义如下：

```
struct thread_info {
    struct task_struct    *task;
    struct exec_domain    *exec_domain;
    __u32                 flags;
    __u32                 status;
    __u32                 cpu;
    int                   preempt_count;
    mm_segment_t          addr_limit;
    struct restart_block  restart_block;
    void                  *sysenter_return;
    int                   uaccess_err;
};
```

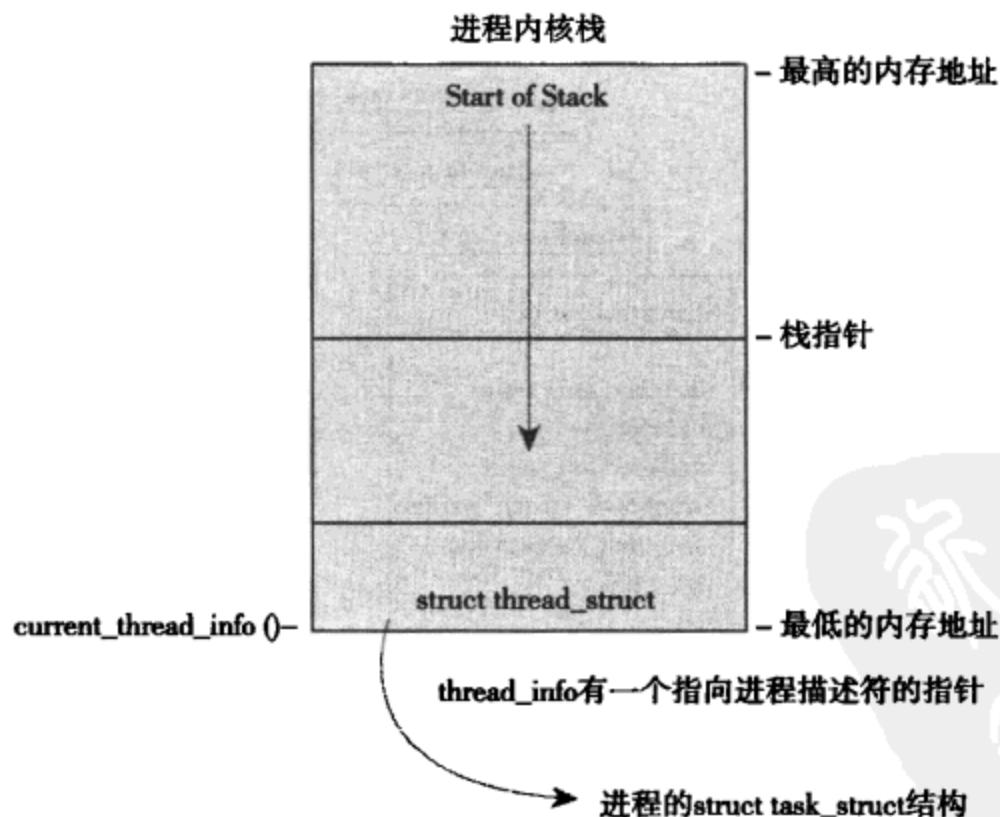


图 3-2 进程描述符和内核栈

① 通过预先分配和重复使用 `task_struct`，可以避免动态分配和释放所带来的资源消耗，还记得吗，第 1 章说过，Unix 的一个特点就是进程创建迅速。——译者注

② 寄存器较弱的体系结构不是引入 `thread_info` 结构的唯一原因。这个新建的结构使在汇编代码中计算其偏移变得非常容易。

每个任务的 `thread_info` 结构在它的内核栈的尾端分配。结构中 `task` 域中存放的是指向该任务实际 `task_struct` 的指针。

3.2.2 进程描述符的存放

内核通过一个唯一的进程标识值 (process identification value) 或 PID 来标识每个进程。PID 是一个数, 表示为 `pid_t` 隐含类型[⊖], 实际上就是一个 `int` 类型。为了与老版本的 Unix 和 Linux 兼容, PID 的最大值默认设置为 32768 (`short int` 短整型的最大值), 尽管这个值也可以增加到高达 400 万 (这受 `<linux/threads.h>` 中所定义 PID 最大值的限制)。内核把每个进程的 PID 存放在它们各自的进程描述符中。

这个最大值很重要, 因为它实际上就是系统中允许同时存在的进程的最大数目。尽管 32768 对于一般的桌面系统足够用了, 但是大型服务器可能需要更多进程。这个值越小, 转一圈就越快, 本来数值大的进程比数值小的进程迟运行, 但这样一来就破坏了这一原则。如果确实需要的话, 可以不考虑与老式系统的兼容, 由系统管理员通过修改 `/proc/sys/kernel/pid_max` 来提高上限。

在内核中, 访问任务通常需要获得指向其 `task_struct` 的指针。实际上, 内核中大部分处理进程的代码都是直接通过 `task_struct` 进行的。因此, 通过 `current` 宏查找到当前正在运行进程的进程描述符的速度就显得尤为重要。硬件体系结构不同, 该宏的实现也不同, 它必须针对专门的硬件体系结构做处理。有的硬件体系结构可以拿出一个专门寄存器来存放指向当前进程 `task_struct` 的指针, 用于加快访问速度。而有些像 x86 这样的体系结构 (其寄存器并不富余), 就只能在内核栈的尾端创建 `thread_info` 结构, 通过计算偏移间接地查找 `task_struct` 结构。

在 x86 系统上, `current` 把栈指针的后 13 个有效位屏蔽掉, 用来计算出 `thread_info` 的偏移。该操作是通过 `current_thread_info()` 函数来完成的。汇编代码如下:

```
movl $-8192, %eax
andl %esp, %eax
```

这里假定栈的大小为 8KB。当 4KB 的栈启用时, 就要用 4096, 而不是 8192。

最后, `current` 再从 `thread_info` 的 `task` 域中提取并返回 `task_struct` 的地址:

```
current_thread_info()->task;
```

对比一下这部分在 PowerPC 上的实现 (IBM 基于 RISC 的现代微处理器), 我们可以发现 PPC 当前的 `task_struct` 是保存在一个寄存器中的。也就是说, 在 PPC 上, `current` 宏只需把 r2 寄存器中的值返回就行了。与 x86 不一样, PPC 有足够多的寄存器, 所以它的实现有这样选择的余地。而访问进程描述符是一个重要的频繁操作, 所以 PPC 的内核开发者觉得完全有必要为此使用一个专门的寄存器。

3.2.3 进程状态

进程描述符中的 `state` 域描述了进程的当前状态 (见图 3-3)。系统中的每个进程都必然处于

⊖ 隐含类型指数据类型的物理表示是未知的或不相关的。

五种进程状态中的一种。该域的值也必为下列五种状态标志之一：

- **TASK_RUNNING (运行)**——进程是可执行的；它或者正在执行，或者在运行队列中等待执行（运行队列将会在第4章中讨论）。这是进程在用户空间中执行的唯一可能的状态；这种状态也可以应用到内核空间中正在执行的进程。
- **TASK_INTERRUPTIBLE (可中断)**——进程正在睡眠（也就是说它被阻塞），等待某些条件的达成。一旦这些条件达成，内核就会把进程状态设置为运行。处于此状态的进程也会因为接收到信号而提前被唤醒并随时准备投入运行。
- **TASK_UNINTERRUPTIBLE (不可中断)**——除了就算是接收到信号也不会被唤醒或准备投入运行外，这个状态与可打断状态相同。这个状态通常在进程必须在等待时不受干扰或等待事件很快会发生时出现。由于处于此状态的任务对信号不做响应，所以较之可中断状态[⊖]，使用得较少。
- **__TASK_TRACED**——被其他进程跟踪的进程，例如通过 `ptrace` 对调试程序进行跟踪。
- **__TASK_STOPPED (停止)**——进程停止执行；进程没有投入运行也不能投入运行。通常这种状态发生在接收到 `SIGSTOP`、`SIGTSTP`、`SIGTTIN`、`SIGTTOU` 等信号的时候。此外，在调试期间接收到任何信号，都会使进程进入这种状态。

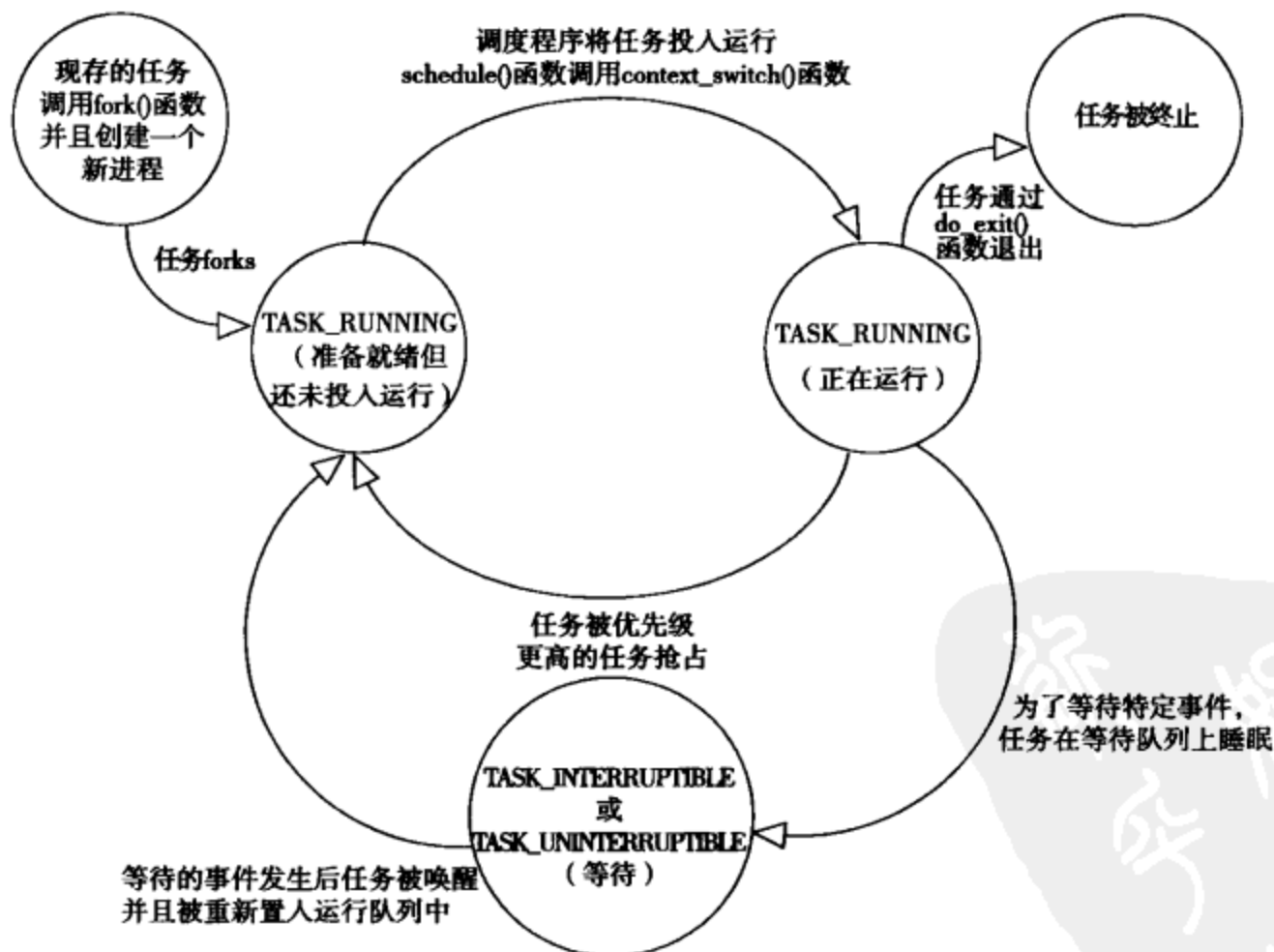


图 3-3 进程状态转化

⊖ 这就是你在执行 `ps(1)` 命令时，看到那些被标为 D 状态而又不能被杀死的进程的原因。由于任务将不响应信号，因此，你不可能给它发送 `SIGKILL` 信号。退一步说，即使有办法，终结这样一个任务也不是明智的选择，因为该任务有可能正在执行重要的操作，甚至还可能持有一个信号量。

3.2.4 设置当前进程状态

内核经常需要调整某个进程的状态。这时最好使用 `set_task_state(task, state)` 函数：

```
set_task_state(task, state);          /* 将任务 task 的状态设置为 state */
```

该函数将指定的进程设置为指定的状态。必要的时候，它会设置内存屏障来强制其他处理器作重新排序。（一般只有在 SMP 系统中有此必要。）否则，它等价于：

```
task->state = state;
```

`set_current_state(state)` 和 `set_task_state(current, state)` 含义是等同的。参看 `<linux/sched.h>` 中对这些相关函数实现的说明。

3.2.5 进程上下文

可执行程序代码是进程的重要组成部分。这些代码从一个可执行文件载入到进程的地址空间执行。一般程序在用户空间执行。当一个程序调执行了系统调用（参见第 5 章）或者触发了某个异常，它就陷入了内核空间。此时，我们称内核“代表进程执行”并处于进程上下文中。在此上下文中 `current` 宏是有效的[⊖]。除非在此间隙有更高优先级的进程需要执行并由调度器做出了相应调整，否则在内核退出的时候，程序恢复在用户空间会继续执行。

系统调用和异常处理程序是对内核明确定义的接口。进程只有通过这些接口才能陷入内核执行——对内核的所有访问都必须通过这些接口。

3.2.6 进程家族树

Unix 系统的进程之间存在一个明显的继承关系，在 Linux 系统中也是如此。所有的进程都是 PID 为 1 的 `init` 进程的后代。内核在系统启动的最后阶段启动 `init` 进程。该进程读取系统的初始化脚本（`initscript`）并执行其他的相关程序，最终完成系统启动的整个过程。

系统中的每个进程必有一个父进程，相应的，每个进程也可以拥有零个或多个子进程。拥有同一个父进程的所有进程被称为兄弟。进程间的关系存放在进程描述符中。每个 `task_struct` 都包含一个指向其父进程 `task_struct`、叫做 `parent` 的指针，还包含一个称为 `children` 的子进程链表。所以，对于当前进程，可以通过下面的代码获得其父进程的进程描述符：

```
struct task_struct *my_parent = current->parent;
```

同样，也可以按以下方式依次访问子进程：

```
struct task_struct *task;
struct list_head *list;

list_for_each(list, &current->children) {
    task = list_entry(list, struct task_struct, sibling);
}
```

⊖ 除了进程上下文，我们将在第 7 章讨论中断上下文。在中断上下文中，系统不代表进程执行，而是执行一个中断处理程序。不会有进程去干扰这些中断处理程序，所以此时不存在进程上下文。

```

        /* task 现在指向当前的某个子进程 */
    }

```

init 进程的进程描述符是作为 init_task 静态分配的。下面的代码可以很好地演示所有进程之间的关系：

```

struct task_struct *task;

for (task = current; task != &init_task; task = task->parent)
    ;
/* task 现在指向 init */

```

实际上，你可以通过这种继承体系从系统的任何一个进程出发查找到任意指定的其他进程。但大多数时候，只需要通过简单的重复方式就可以遍历系统中的所有进程。这非常容易做到，因为任务队列本来就是一个双向的循环链表。对于给定的进程，获取链表中的下一个进程：

```
list_entry(task->tasks.next, struct task_struct, tasks)
```

获取前一个进程的方法与之相同：

```
list_entry(task->tasks.prev, struct task_struct, tasks)
```

这两个例程分别通过 next_task(task) 宏和 prev_task(task) 宏实现。而实际上，for_each_process(task) 宏提供了依次访问整个任务队列的能力。每次访问，任务指针都指向链表中的下一个元素：

```

struct task_struct *task;

for_each_process(task) {
    /* 它打印出每一个任务的名称和 PID */
    printk("%s[%d]\n", task->comm, task->pid);
}

```

特别提醒 在一个拥有大量进程的系统中通过重复来遍历所有的进程代价是很大的。因此，如果没有充足的理由（或者别无他法），别这样做。

3.3 进程创建

Unix 的进程创建很特别。许多其他的操作系统都提供了产生 (spawn) 进程的机制，首先在新的地址空间里创建进程，读入可执行文件，最后开始执行。Unix 采用了与众不同的实现方式，它把上述步骤分解到两个单独的函数中去执行：fork() 和 exec() ^①。首先，fork() 通过拷贝当前进程创建一个子进程。子进程与父进程的区别仅仅在于 PID（每个进程唯一）、PPID（父进程的进程号，子进程将其设置为被拷贝进程的 PID）和某些资源和统计量（例如，挂起的信号，它没有必要被继承）。exec() 函数负责读取可执行文件并将其载入地址空间开始运行。把这两个函数组合起来使用的效果跟其他系统使用的单一函数的效果相似。

^① exec() 在这里指所有 exec() 一族的函数。内核实现了 execve() 函数，在此基础上，还实现了 execlp()、execle()、execv() 和 execvp()。

3.3.1 写时拷贝

传统的 `fork()` 系统调用直接把所有的资源复制给新创建的进程。这种实现过于简单并且效率低下，因为它拷贝的数据也许并不共享，更糟的情况是，如果新进程打算立即执行一个新的映像，那么所有的拷贝都将前功尽弃。Linux 的 `fork()` 使用写时拷贝 (copy-on-write) 页实现。写时拷贝是一种可以推迟甚至免除拷贝数据的技术。内核此时并不复制整个进程地址空间，而是让父进程和子进程共享同一个拷贝。

只有在需要写入的时候，数据才会被复制，从而使各个进程拥有各自的拷贝。也就是说，资源的复制只有在需要写入的时候才进行，在此之前，只是以只读方式共享。这种技术使地址空间上的页的拷贝被推迟到实际发生写入的时候才进行。在页根本不会被写入的情况下（举例来说，`fork()` 后立即调用 `exec()`）它们就无须复制了。

`fork()` 的实际开销就是复制父进程的页表以及给子进程创建唯一的进程描述符。在一般情况下，进程创建后都会马上运行一个可执行的文件，这种优化可以避免拷贝大量根本就不会被使用的数据（地址空间里常常包含数十兆的数据）。由于 Unix 强调进程快速执行的能力，所以这个优化是很重要的。

3.3.2 `fork()`

Linux 通过 `clone()` 系统调用实现 `fork()`。这个调用通过一系列的参数标志来指明父、子进程需要共享的资源（关于这些标志更多的信息请参考本章后面 3.4 节）。`fork()`、`vfork()` 和 `__clone()` 库函数都根据各自需要的参数标志去调用 `clone()`，然后由 `clone()` 去调用 `do_fork()`。

`do_fork` 完成了创建中的大部分工作，它的定义在 `kernel/fork.c` 文件中。该函数调用 `copy_process()` 函数，然后让进程开始运行。`copy_process()` 函数完成的工作很有意思：

- 1) 调用 `dup_task_struct()` 为新进程创建一个内核栈、`thread_info` 结构和 `task_struct`，这些值与当前进程的值相同。此时，子进程和父进程的描述符是完全相同的。
- 2) 检查并确保新建这个子进程后，当前用户所拥有的进程数目没有超出给它分配的资源限制。
- 3) 子进程着手使自己与父进程区别开来。进程描述符内的许多成员都要被清 0 或设为初始值。那些不是继承而来的进程描述符成员，主要是统计信息。`task_struct` 中的大多数数据都依然未被修改。
- 4) 子进程的状态被设置为 `TASK_UNINTERRUPTIBLE`，以保证它不会投入运行。
- 5) `copy_process()` 调用 `copy_flags()` 以更新 `task_struct` 的 `flags` 成员。表明进程是否拥有超级用户权限的 `PF_SUPERPRIV` 标志被清 0。表明进程还没有调用 `exec()` 函数的 `PF_FORKNOEXEC` 标志被设置。
- 6) 调用 `alloc_pid()` 为新进程分配一个有效的 PID。
- 7) 根据传递给 `clone()` 的参数标志，`copy_process()` 拷贝或共享打开的文件、文件系统信息、信号处理函数、进程地址空间和命名空间等。在一般情况下，这些资源会被给定进程的所有线程共享；否则，这些资源对每个进程是不同的，因此被拷贝到这里。

8) 最后, `copy_process()` 做扫尾工作并返回一个指向子进程的指针。

再回到 `do_fork()` 函数, 如果 `copy_process()` 函数成功返回, 新创建的子进程被唤醒并让其投入运行。内核有意选择子进程首先执行^①。因为一般子进程都会马上调用 `exec()` 函数, 这样可以避免写时拷贝的额外开销, 如果父进程首先执行的话, 有可能会开始向地址空间写入。

3.3.3 vfork()

除了不拷贝父进程的页表项外, `vfork()` 系统调用和 `fork()` 的功能相同。子进程作为父进程的一个单独的线程在它的地址空间里运行, 父进程被阻塞, 直到子进程退出或执行 `exec()`。子进程不能向地址空间写入。在过去的 3BSD 时期, 这个优化是很有意义的, 那时并未使用写时拷贝页来实现 `fork()`。现在由于在执行 `fork()` 时引入了写时拷贝页并且明确了子进程先执行, `vfork()` 的好处就仅限于不拷贝父进程的页表项了。如果 Linux 将来 `fork()` 有了写时拷贝页表项, 那么 `vfork()` 就彻底没用了^②。另外由于 `vfork()` 语意非常微妙 (试想, 如果 `exec()` 调用失败会发生什么), 所以理想情况下, 系统最好不要调用 `vfork()`, 内核也不用实现它。完全可以把 `vfork()` 实现成一个普普通通的 `fork()`——实际上, Linux 2.2 以前都是这么做的。

`vfork()` 系统调用的实现是通过向 `clone()` 系统调用传递一个特殊标志来进行的。

- 1) 在调用 `copy_process()` 时, `task_struct` 的 `vfork_done` 成员被设置为 NULL。
- 2) 在执行 `do_fork()` 时, 如果给定特别标志, 则 `vfork_done` 会指向一个特定地址。
- 3) 子进程先开始执行后, 父进程不是马上恢复执行, 而是一直等待, 直到子进程通过 `vfork_done` 指针向它发送信号。
- 4) 在调用 `mm_release()` 时, 该函数用于进程退出内存地址空间, 并且检查 `vfork_done` 是否为空, 如果不为空, 则会向父进程发送信号。
- 5) 回到 `do_fork()`, 父进程醒来并返回。

如果一切执行顺利, 子进程在新的地址空间里运行而父进程也恢复了在原地址空间的运行。这样, 开销确实降低了, 不过它的实现并不是优良的。

3.4 线程在 Linux 中的实现

线程机制是现代编程技术中常用的一种抽象概念。该机制提供了在同一程序内共享内存地址空间运行的一组线程。这些线程还可以共享打开的文件和其他资源。线程机制支持并发程序设计技术 (concurrent programming), 在多处理器系统上, 它也能保证真正的并行处理 (parallelism)。

Linux 实现线程的机制非常独特。从内核的角度来说, 它并没有线程这个概念。Linux 把所有的线程都当做进程来实现。内核并没有准备特别的调度算法或是定义特别的数据结构来表征线程。相反, 线程仅仅被视为一个与其他进程共享某些资源的进程。每个线程都拥有唯一隶属于自己的 `task_struct`, 所以在内核中, 它看起来就像是一个普通的进程 (只是线程和其他一些进程共享某些资源, 如地址空间)。

① 有趣的是, 虽然想让子进程先运行, 但是并非总能如此。

② 有补丁可以帮助 Linux 完成该功能。这种特性很可能找到自己的途径而进入 Linux 主内核。

上述线程机制的实现与 Microsoft Windows 或是 Sun Solaris 等操作系统的实现差异非常大。这些系统都在内核中提供了专门支持线程的机制（这些系统常常把线程称作轻量级进程（lightweight processes））。“轻量级进程”这种叫法本身就概括了 Linux 在此处与其他系统的差异。在其他的系统中，相较于重量级的进程，线程被抽象成一种耗费较少资源，运行迅速的执行单元。而对于 Linux 来说，它只是一种进程间共享资源的手段（Linux 的进程本身就够轻量级了）^①。举个例子来说，假如我们有一个包含四个线程的进程，在提供专门线程支持的系统中，通常会有一个包含指向四个不同线程的指针的进程描述符。该描述符负责描述像地址空间、打开的文件这样的共享资源。线程本身再去描述它独占的资源。相反，Linux 仅仅创建四个进程并分配四个普通的 `task_struct` 结构。建立这四个进程时指定他们共享某些资源，这是相当高雅的做法。

3.4.1 创建线程

线程的创建和普通进程的创建类似，只不过在调用 `clone()` 的时候需要传递一些参数标志来指明需要共享的资源：

```
clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);
```

上面的代码产生的结果和调用 `fork()` 差不多，只是父子俩共享地址空间、文件系统资源、文件描述符和信号处理程序。换个说法就是，新建的进程和它的父进程就是流行的所谓线程。

对比一下，一个普通的 `fork()` 的实现是：

```
clone(SIGCHLD, 0);
```

而 `vfork()` 的实现是：

```
clone(CLONE_VFORK | CLONE_VM | SIGCHLD, 0);
```

传递给 `clone()` 的参数标志决定了新创建进程的行为方式和父子进程之间共享的资源种类。表 3-1 列举了这些 `clone()` 用到的参数标志以及它们的作用，这些是在 `<linux/sched.h>` 中定义的。

表 3-1 `clone()` 参数标志

参数标志	含 义
<code>CLONE_FILES</code>	父子进程共享打开的文件
<code>CLONE_FS</code>	父子进程共享文件系统信息
<code>CLONE_IDLETASK</code>	将 PID 设置为 0（只供 idle 进程使用）
<code>CLONE_NEWNS</code>	为子进程创建新的命名空间
<code>CLONE_PARENT</code>	指定子进程与父进程拥有同一个父进程
<code>CLONE_PTRACE</code>	继续调试子进程
<code>CLONE_SETTID</code>	将 TID 回写至用户空间
<code>CLONE_SETTLS</code>	为子进程创建新的 TLS
<code>CLONE_SIGHAND</code>	父子进程共享信号处理函数及被阻断的信号

^① 作为一个例子，创建 Linux 进程所花时间和创建其他操作系统（尤其是线程）所花时间的比较测评结果非常好。

(续)

参数标志	含 义
CLONE_SYSVSEM	父子进程共享 System V SEM_UNDO 语义
CLONE_THREAD	父子进程放入相同的线程组
CLONE_VFORK	调用 vfork(), 所以父进程准备睡眠等待子进程将其唤醒
CLONE_UNTRACED	防止跟踪进程在子进程上强制执行 CLONE_PTRACE
CLONE_STOP	以 TASK_STOPPED 状态开始进程
CLONE_SETTLS	为子进程创建新的 TLS(thread-local storage)
CLONE_CHILD_CLEARTID	清除子进程的 TID
CLONE_CHILD_SETTID	设置子进程的 TID
CLONE_PARENT_SETTID	设置父进程的 TID
CLONE_VM	父子进程共享地址空间

3.4.2 内核线程

内核经常需要在后台执行一些操作。这种任务可以通过内核线程 (kernel thread) 完成——独立运行在内核空间的标准进程。内核线程和普通的进程间的区别在于内核线程没有独立的地址空间 (实际上指向地址空间的 mm 指针被设置为 NULL)。它们只在内核空间运行, 从来不切换到用户空间去。内核进程和普通进程一样, 可以被调度, 也可以被抢占。

Linux 确实会把一些任务交给内核线程去做, 像 flush 和 ksofirqd 这些任务就是明显的例子。在装有 Linux 系统的机子上运行 ps -ef 命令, 你可以看到内核线程, 有很多! 这些线程在系统启动时由另外一些内核线程创建。实际上, 内核线程也只能由其他内核线程创建。内核是通过从 kthreadd 内核进程中衍生出所有新的内核线程来自动处理这一点的。在 <linux/kthread.h> 中申明有接口, 于是, 从现有内核线程中创建一个新的内核线程的方法如下:

```
struct task_struct *kthread_create(int (*threadfn)(void *data),
                                   void *data,
                                   const char namefmt[],
                                   ...)
```

新的任务是由 kthread 内核进程通过 clone() 系统调用而创建的。新的进程将运行 threadfn 函数, 给其传递的参数为 data。进程会被命名为 namefmt, namefmt 接受可变参数列表类似于 printf() 的格式化参数。新创建的进程处于不可运行状态, 如果不通过调用 wake_up_process() 明确地唤醒它, 它不会主动运行。创建一个进程并让它运行起来, 可以通过调用 kthread_run() 来达到:

```
struct task_struct *kthread_run(int (*threadfn)(void *data),
                                void *data,
                                const char namefmt[],
                                ...)
```

这个例程是以宏实现的, 只是简单地调用了 kthread_create() 和 wake_up_process():

```

#define kthread_run(threadfn, data, namefmt, ...) \
({ \
    struct task_struct *k; \
 \
    k = kthread_create(threadfn, data, namefmt, ## __VA_ARGS__); \
    if (!IS_ERR(k)) \
        wake_up_process(k); \
 \
    k; \
})

```

内核线程启动后就一直运行直到调用 `do_exit()` 退出，或者内核的其他部分调用 `kthread_stop()` 退出，传递给 `kthread_stop()` 的参数为 `kthread_create()` 函数返回的 `task_struct` 结构的地址：

```
int kthread_stop(struct task_struct *k)
```

我们将在以后的内容中详细讨论具体的内核线程。

3.5 进程终结

虽然让人伤感，但进程终归是要终结的。当一个进程终结时，内核必须释放它所占有的资源并把这一不幸告知其父进程。

一般来说，进程的析构是自身引起的。它发生在进程调用 `exit()` 系统调用时，既可能显式地调用这个系统调用，也可能隐式地从某个程序的主函数返回（其实 C 语言编译器会在 `main()` 函数的返回点后面放置调用 `exit()` 的代码）。当进程接受到它既不能处理也不能忽略的信号或异常时，它还可能被动地终结。不管进程是怎么终结的，该任务大部分都要靠 `do_exit()`（定义于 `kernel/exit.c`）来完成，它要做下面这些烦琐的工作：

- 1) 将 `task_struct` 中的标志成员设置为 `PF_EXITING`。
- 2) 调用 `del_timer_sync()` 删除任一内核定时器。根据返回的结果，它确保没有定时器在排队，也没有定时器处理程序在运行。
- 3) 如果 BSD 的进程记账功能是开启的，`do_exit()` 调用 `acct_update_integrals()` 来输出记账信息。
- 4) 然后调用 `exit_mm()` 函数释放进程占用的 `mm_struct`，如果没有别的进程使用它们（也就是说，这个地址空间没有被共享），就彻底释放它们。
- 5) 接下来调用 `sem_exit()` 函数。如果进程排队等候 IPC 信号，它则离开队列。
- 6) 调用 `exit_files()` 和 `exit_fs()`，以分别递减文件描述符、文件系统数据的引用计数。如果其中某个引用计数的数值降为零，那么就代表没有进程在使用相应的资源，此时可以释放。
- 7) 接着把存放在 `task_struct` 的 `exit_code` 成员中的任务退出代码置为由 `exit()` 提供的退出代码，或者去完成任何其他由内核机制规定的退出动作。退出代码存放在这里供父进程随时检索。
- 8) 调用 `exit_notify()` 向父进程发送信号，给予进程重新找养父，养父为线程组中的其他线程或者为 `init` 进程，并把进程状态（存放在 `task_struct` 结构的 `exit_state` 中）设成 `EXIT_ZOMBIE`。
- 9) `do_exit()` 调用 `schedule()` 切换到新的进程（参看第 4 章）。因为处于 `EXIT_ZOMBIE` 状态

的进程不会再被调度，所以这是进程所执行的最后一段代码。do_exit() 永不返回。

至此，与进程相关联的所有资源都被释放掉了（假设该进程是这些资源的唯一使用者）。进程不可运行（实际上也没有地址空间让它运行）并处于 EXIT_ZOMBIE 退出状态。它占用的所有内存就是内核栈、thread_info 结构和 task_struct 结构。此时进程存在的唯一目的就是向它的父进程提供信息。父进程检索到信息后，或者通知内核那是无关的信息后，由进程所持有的剩余内存被释放，归还给系统使用。

3.5.1 删除进程描述符

在调用了 do_exit() 之后，尽管线程已经僵死不能再运行了，但是系统还保留了它的进程描述符。前面说过，这样做可以让系统有办法在子进程终结后仍能获得它的信息。因此，进程终结时所需的清理工作和进程描述符的删除被分开执行。在父进程获得已终结的子进程的信息后，或者通知内核它并不关注那些信息后，子进程的 task_struct 结构才被释放。

wait() 这一族函数都是通过唯一（但是很复杂）的一个系统调用 wait4() 来实现的。它的标准动作是挂起调用它的进程，直到其中的一个子进程退出，此时函数会返回该子进程的 PID。此外，调用该函数时提供的指针会包含子函数退出时的退出代码。

当最终需要释放进程描述符时，release_task() 会被调用，用以完成以下工作：

1) 它调用 __exit_signal(), 该函数调用 _unhash_process(), 后者又调用 detach_pid() 从 pidhash 上删除该进程，同时也要从任务列表中删除该进程。

2) _exit_signal() 释放目前僵死进程所使用的所有剩余资源，并进行最终统计和记录。

3) 如果这个进程是线程组最后一个进程，并且领头进程已经死掉，那么 release_task() 就要通知僵死的领头进程的父进程。

4) release_task() 调用 put_task_struct() 释放进程内核栈和 thread_info 结构所占的页，并释放 task_struct 所占的 slab 高速缓存。

至此，进程描述符和所有进程独享的资源就全部释放掉了。

3.5.2 孤儿进程造成的进退维谷

如果父进程在子进程之前退出，必须有机制来保证子进程能找到一个新父亲，否则这些成为孤儿的进程就会在退出时永远处于僵死状态，白白地耗费内存。前面的部分已经有所暗示，对于这个问题，解决方法是给予进程在当前线程组内找一个线程作为父亲，如果不行，就让 init 做它们的父进程。在 do_exit() 中会调用 exit_notify(), 该函数会调用 forget_original_parent(), 而后者会调用 find_new_reaper() 来执行寻父过程：

```
static struct task_struct *find_new_reaper(struct task_struct *father)
{
    struct pid_namespace *pid_ns = task_active_pid_ns(father);
    struct task_struct *thread;

    thread = father;
    while_each_thread(father, thread) {
```

```

        if (thread->flags & PF_EXITING)
            continue;
        if (unlikely(pid_ns->child_reaper == father))
            pid_ns->child_reaper = thread;
        return thread;
    }

    if (unlikely(pid_ns->child_reaper == father)) {
        write_unlock_irq(&tasklist_lock);
        if (unlikely(pid_ns == &init_pid_ns))
            panic("Attempted to kill init!");

        zap_pid_ns_processes(pid_ns);
        write_lock_irq(&tasklist_lock);
        /*
         * We can not clear ->child_reaper or leave it alone.
         * There may be stealth EXIT_DEAD tasks on ->children,
         * forget_original_parent() must move them somewhere.
         */
        pid_ns->child_reaper = init_pid_ns.child_reaper;
    }
    return pid_ns->child_reaper;
}

```

这段代码试图找到进程所在的线程组内的其他进程。如果线程组内没有其他的进程，它就找到并返回的是 `init` 进程。现在，给予进程找到合适的养父进程了，只需要遍历所有子进程并为它们设置新的父进程：

```

reaper = find_new_reaper(father);
list_for_each_entry_safe(p, n, &father->children, sibling) {
    p->real_parent = reaper;
    if (p->parent == father) {
        BUG_ON(p->ptrace);
        p->parent = p->real_parent;
    }
    reparent_thread(p, father);
}

```

然后调用 `ptrace_exit_finish()` 同样进行新的寻父过程，不过这次是给 `ptraced` 的子进程寻找父亲。

```

void exit_ptrace(struct task_struct *tracer)
{
    struct task_struct *p, *n;
    LIST_HEAD(ptrace_dead);

    write_lock_irq(&tasklist_lock);
    list_for_each_entry_safe(p, n, &tracer->ptraced, ptrace_entry) {
        if (__ptrace_detach(tracer, p))
            list_add(&p->ptrace_entry, &ptrace_dead);
    }
}

```



```
    }  
    write_unlock_irq(&tasklist_lock);  
  
    BUG_ON(!list_empty(&tracer->ptraced));  
  
    list_for_each_entry_safe(p, n, &ptrace_dead, ptrace_entry) {  
        list_del_init(&p->ptrace_entry);  
        release_task(p);  
    }  
}
```

这段代码遍历了两个链表：子进程链表和 ptrace 子进程链表，给每个子进程设置新的父进程。这两个链表同时存在的原因很有意思，它也是 2.6 内核的一个新特性。当一个进程被跟踪时，它的临时父亲设定为调试进程。此时如果它的父进程退出了，系统会为它和他的所有兄弟重新找一个父进程。在以前的内核中，这就需要遍历系统所有的进程来找这些子进程。现在的解决办法是在一个单独的被 ptrace 跟踪的子进程链表中搜索相关的兄弟进程——用两个相对较小的链表减轻了遍历带来的消耗。

一旦系统为进程成功地找到和设置了新的父进程，就不会再有出现驻留僵死进程的危险了。init 进程会例行调用 wait() 来检查其子进程，清除所有与其相关的僵死进程。

3.6 小结

在本章中，我们考察了操作系统中的核心概念——进程。我们也讨论了进程的一般特性，它为何如此重要，以及进程与线程之间的关系。然后，讨论了 Linux 如何存放和表示进程（用 task_struct 和 thread_info），如何创建进程（通过 fork()，实际上最终是 clone()），如何把新的执行映像装入到地址空间（通过 exec() 系统调用族），如何表示进程的层次关系，父进程又是如何收集其后代的信息（通过 wait() 系统调用族），以及进程最终如何消亡（强制或自愿地调用 exit()）。进程是一个非常基础、非常关键的抽象概念，位于每一种现代操作系统的核心位置，也是我们拥有操作系统（用来运行程序）的最终原因。

第 4 章讨论进程调度，内核以这种微妙而有趣的方式来决定哪个进程运行，何时运行，以何种顺序运行。



第④章

进程调度

第3章讨论了进程，它在操作系统看来是程序的运行态表现形式。本章将讨论进程调度程序，它是确保进程能有效工作的一个内核子系统。

调度程序负责决定将哪个进程投入运行，何时运行以及运行多长时间。进程调度程序（常常简称调度程序）可看做在可运行态进程之间分配有限的处理器时间资源的内核子系统。调度程序是像 Linux 这样的多任务操作系统的基础。只有通过调度程序的合理调度，系统资源才能最大限度地发挥作用，多进程才会有并发执行的效果。

调度程序没有太复杂的原理。最大限度地利用处理器时间的原则是，只要有可以执行的进程，那么就总会有进程正在执行。但是只要系统中可运行的进程的数目比处理器的个数多，就注定某一给定时刻会有一些进程不能执行。这些进程在等待运行。在一组处于可运行状态的进程中选择一个来执行，是调度程序所需完成的基本工作。

4.1 多任务

多任务操作系统就是能同时并发地交互执行多个进程的操作系统。在单处理器机器上，这会产生多个进程在同时运行的幻觉。在多处理器机器上，这会使多个进程在不同的处理机上真正同时、并行地运行。无论在单处理器或者多处理器机器上，多任务操作系统都能使多个进程处于堵塞或者睡眠状态，也就是说，实际上不被投入执行，直到工作确实就绪。这些任务尽管位于内存，但并不处于可运行状态。相反，这些进程利用内核阻塞自己，直到某一事件（键盘输入、网络数据、过一段时间等）发生。因此，现代 Linux 系统也许有 100 个进程在内存，但是只有一个处于可运行状态。

多任务系统可以划分为两类：非抢占式多任务（cooperative multitasking）和抢占式多任务（preemptive multitasking）。像所有 Unix 的变体和许多其他现代操作系统一样，Linux 提供了抢占式的多任务模式。在此模式下，由调度程序来决定什么时候停止一个进程的运行，以便其他进程能够得到执行机会。这个强制的挂起动作就叫做抢占（preemption）。进程在被抢占之前能够运行的时间是预先设置好的，而且有一个专门的名字，叫进程的时间片（timeslice）。时间片实际上就是分配给每个可运行进程的处理器时间段。有效管理时间片能使调度程序从系统全局的角度做出调度决定，这样做还可以避免个别进程独占系统资源。当今众多现代操作系统对程序运行都采用了动态时间片计算的方式，并且引入了可配置的计算策略。不过我们将看到，Linux 独一无二的“公平”调度程度本身并没有采取时间片来达到公平调度。

相反，在非抢占式多任务模式下，除非进程自己主动停止运行，否则它会一直执行。进程主

动挂起自己的操作称为让步 (yielding)。理想情况下, 进程通常做出让步, 以便让每个可运行进程享有足够的处理器时间。但这种机制有很多缺点: 调度程序无法对每个进程该执行多长时间做出统一规定, 所以进程独占的处理器时间可能超出用户的预料; 更糟的是, 一个决不做出让步的悬挂进程就能使系统崩溃。幸运的是, 近 20 年以来, 绝大部分的操作系统的设计都采用了抢占式多任务——除了 Mac OS 9 (以及其前身)、还有 Windows 3.1 (以及其前身) 这些出名且麻烦的异端以外。毫无疑问, Unix 从一开始就采用的是抢先式的多任务。

4.2 Linux 的进程调度

从 1991 年 Linux 的第 1 版到后来的 2.4 内核系列, Linux 的调度程序都相当简陋, 设计近乎原始。当然它很容易理解, 但是它在众多可运行进程或者多处理器的环境下都难以胜任。

正因为如此, 在 Linux 2.5 开发系列的内核中, 调度程序做了大手术。开始采用了一种叫做 $O(1)$ 调度程序的新调度程序——它是因为其算法的行为而得名的^①。它解决了先前版本 Linux 调度程序的许多不足, 引入了许多强大的新特性和性能特征。这里主要要感谢静态时间片算法和针对每一处理器的运行队列, 它们帮助我们摆脱了先前调度程序设计上的限制。

$O(1)$ 调度器虽然在拥有数以十计 (不是数以百计) 的多处理器的环境下尚能表现出近乎完美的性能和可扩展性, 但是时间证明该调度算法对于调度那些响应时间敏感的程序却有一些先天不足。这些程序我们称其为交互进程——它无疑包括了所有需要用户交互的程序。正因为如此, $O(1)$ 调度程序虽然对于大服务器的工作负载很理想, 但是在有很多交互程序要运行的桌面系统上则表现不佳, 因为其缺少交互进程。自 2.6 内核系统开发初期, 开发人员为了提高对交互程序的调度性能引入了新的进程调度算法。其中最为著名的是“反转楼梯最后期限调度算法 (Rotating Staircase Deadline scheduler)” (RSDL), 该算法吸取了队列理论, 将公平调度的概念引入了 Linux 调度程序。并且最终在 2.6.23 内核版本中替代了 $O(1)$ 调度算法, 它此刻被称为“完全公平调度算法”, 或者简称 CFS。

本章将讲解调度程序设计的基础和完全公平调度程序如何运用、如何设计、如何实现以及它与相关的系统调用。我们当然也会讲解 $O(1)$ 调度程序, 因为它毕竟是经典 Unix 调度程序模型的实现方式。

4.3 策略

策略决定调度程序在何时让什么进程运行。调度器的策略往往就决定系统的整体印象, 并且, 还要负责优化使用处理器时间。无论从哪个方面来看, 它都是至关重要的。

4.3.1 I/O 消耗型和处理器消耗型的进程

进程可以被分为 I/O 消耗型和处理器消耗型。前者指进程的大部分时间用来提交 I/O 请求或是等待 I/O 请求。因此, 这样的进程经常处于可运行状态, 但通常都是运行短短的一会儿, 因为

^① $O(1)$ 用的是大 O 表示法。简而言之, 它是指不管输入有多大, 调度程序都可以在恒定时间内完成工作。第 6 章是一份完整的大 O 表示法说明。

它在等待更多的 I/O 请求时最后总会阻塞（这里所说的 I/O 是指任何类型的可阻塞资源，比如键盘输入，或者是网络 I/O）。举例来说，多数用户图形界面程序（GUI）都属于 I/O 密集型，即便它们从不读取或者写入磁盘，它们也会在多数时间里都在等待来自鼠标或者键盘的用户交互操作。

相反，处理器耗费型进程把时间大多用在执行代码上。除非被抢占，否则它们通常都一直不停地运行，因为它们没有太多的 I/O 需求。但是，因为它们不属于 I/O 驱动类型，所以从系统响应速度考虑，调度器不应该经常让它们运行。对于这类处理器消耗型的进程，调度策略往往是尽量降低它们的调度频率，而延长其运行时间。处理器消耗型进程的极端例子就是无限循环地执行。更具代表性的例子是那些执行大量数学计算的程序，如 `sshkeygen` 或者 `MATLAB`。

当然，这种划分方法并非是绝对的。进程可以同时展示这两种行为：比如，X Window 服务器既是 I/O 消耗型，也是处理器消耗型。还有些进程可以是 I/O 消耗型，但属于处理器消耗型活动的范围。其典型的例子就是字处理器，其通常坐以等待键盘输入，但在任一时刻可能又粘住处理器疯狂地进行拼写检查或者宏计算。

调度策略通常要在两个矛盾的目标中间寻找平衡：进程响应迅速（响应时间短）和最大系统利用率（高吞吐量）。为了满足上述需求，调度程序通常采用一套非常复杂的算法来决定最值得运行的进程投入运行，但是它往往并不保证低优先级进程会被公平对待。Unix 系统的调度程序更倾向于 I/O 消耗型程序，以提供更好的程序响应速度。Linux 为了保证交互式应用和桌面系统的性能，所以对进程的响应做了优化（缩短响应时间），更倾向于优先调度 I/O 消耗型进程。虽然如此，但在下面你会看到，调度程序也并未忽略处理器消耗型的进程。

4.3.2 进程优先级

调度算法中最基本的一类就是基于优先级的调度。这是一种根据进程的价值和其对处理器时间的需求来对进程分级的想法。通常做法是（其并未被 Linux 系统完全采用）优先级高的进程先运行，低的后运行，相同优先级的进程按轮转方式进行调度（一个接一个，重复进行）。在某些系统中，优先级高的进程使用的时间片也较长。调度程序总是选择时间片未用尽而且优先级最高的进程运行。用户和系统都可以通过设置进程的优先级来影响系统的调度。

Linux 采用了两种不同的优先级范围。第一种是用 `nice` 值，它的范围是从 -20 到 +19，默认值为 0；越大的 `nice` 值意味着更低的优先级——`nice` 似乎意味着你对系统中的其他进程更“优待”。相比高 `nice` 值（低优先级）的进程，低 `nice` 值（高优先级）的进程可以获得更多的处理器时间。`nice` 值是所有 Unix 系统中的标准化的概念——但不同的 Unix 系统由于调度算法不同，因此 `nice` 值的运用方式有所差异。比如一些基于 Unix 的操作系统，如 Mac OS X，进程的 `nice` 值代表分配给进程的时间片的绝对值；而 Linux 系统中，`nice` 值则代表时间片的比例。你可以通过 `ps-el` 命令查看系统中的进程列表，结果中标记 `NI` 的一列就是进程对应的 `nice` 值。

第二种范围是实时优先级，其值是可配置的，默认情况下它的变化范围是从 0 到 99（包括 0 和 99）。与 `nice` 值意义相反，越高的实时优先级数值意味着进程优先级越高。任何实时进程的优先级都高于普通的进程，也就是说实时优先级和 `nice` 优先级处于互不相交的两个范畴。Linux 实时优先级的实现参考了 Unix 相关标准——特别是 POSIX.1b。大部分现代的 Unix 操作系统也都提供类似的机制。你可以通过命令：


```
ps -eo state,uid,pid,ppid,rtprio,time,comm.
```

查看到你系统中的进程列表，以及它们对应的实时优先级（位于 RTPRIO 列下），其中如果有进程对应列显示“-”，则说明它不是实时进程。

4.3.3 时间片

时间片^①是一个数值，它表明进程在被抢占前所能持续运行的时间。调度策略必须规定一个默认的时间片，但这并不是件简单的事。时间片过长会导致系统对交互的响应表现欠佳，让人觉得系统无法并发执行应用程序；时间片太短会明显增大进程切换带来的处理器耗时，因为肯定会有相当一部分系统时间用在进程切换上，而这些进程能够用来运行的时间片却很短。此外，I/O 消耗型和处理器消耗型的进程之间的矛盾在这里也再次显露出来：I/O 消耗型不需要长的时间片，而处理器消耗型的进程则希望越长越好（比如这样可以让它们的高速缓存命中率更高）。

从上面的争论中可以看出，任何长时间片都将导致系统交互表现欠佳。很多操作系统中都特别重视这一点，所以默认的时间片很短，如 10ms。但是 Linux 的 CFS 调度器并没有直接分配时间片到进程，它是将处理器的使用比划分给了进程。这样一来，进程所获得的处理器时间其实是和系统负载密切相关的。这个比例进一步还会受进程 nice 值的影响，nice 值作为权重将调整进程所使用的处理器时间使用比。具有更高 nice 值（更低优先权）的进程将被赋予低权重，从而丧失一小部分的处理器使用比；而具有更小 nice 值（更高优先级）的进程则会被赋予高权重，从而抢得更多的处理器使用比。

像前面所说的，Linux 系统是抢占式的。当一个进程进入可运行态，它就被准许投入运行。在多数操作系统中，是否要将一个进程立刻投入运行（也就是抢占当前进程），是完全由进程优先级和是否有时间片决定的。而在 Linux 中使用新的 CFS 调度器，其抢占时机取决于新的可运行程序消耗了多少处理器使用比。如果消耗的使用比比当前进程小，则新进程立刻投入运行，抢占当前进程。否则，将推迟其运行。

4.3.4 调度策略的活动

想象下面这样一个系统，它拥有两个可运行的进程：一个文字编辑程序和一个视频编码程序。文字编辑程序显然是 I/O 消耗型的，因为它大部分时间都在等待用户的键盘输入（无论用户的输入速度有多快，都不可能赶上处理的速度）。用户总是希望按下键系统就能马上响应。相反，视频编码程序是处理器消耗型的。除了最开始从磁盘上读出原始数据流和最后把处理好的视频输出外，程序所有的时间都用来对原始数据进行视频编码，处理器很轻易地被 100% 使用。它对什么时间开始运行没有太严格的要求——用户几乎分辨不出也并不关心它到底是立刻就运行还是半秒钟以后才开始的。当然，它完成得越早越好，至于所花时间并不是我们关注的主要问题。

^① 在其他系统中，时间片有时也称为量子（quantum）或处理器片（processor slice）。但 Linux 把它叫做时间片，因此你也最好这样叫。

在这样的场景中，理想情况是调度器应该给予文本编辑程序相比视频编码程序更多的处理器时间，因为它属于交互式应用。对文本编辑器而言，我们有两个目标。第一是我们希望系统给它更多的处理器时间，这并非因为它需要更多的处理器时间（其实它不需要），是因为我们希望它在需要时总是能得到处理器；第二是我们希望文本编辑器能在其被唤醒时（也就是当用户打字时）抢占视频解码程序。这样才能确保文本编辑器具有很好的交互性能，以便能响应用户输入。在多数操作系统中，上述目标的达成是要依靠系统分配给文本编辑器比视频解码程序更高的优先级和更多的时间片。先进的操作系统可以自动发现文本编辑器是交互性程序，从而自动地完成上述分配动作。Linux 操作系统同样需要追求上述目标，但是它采用不同方法。它不再通过给文本编辑器分配给定的优先级和时间片，而是分配一个给定的处理器使用比。假如文本编辑器和视频解码程序是仅有的两个运行进程，并且又具有同样的 nice 值，那么处理器的使用比将都是 50%——它们平分了处理器时间。但因为文本编辑器将更多的时间用于等待用户输入，因此它肯定不会用到处理器的 50%。同时，视频解码程序无疑将能有机会用到超过 50% 的处理器时间，以便它能更快速地完成解码任务。

这里关键的问题是，当文本编辑器程序被唤醒时将发生什么。我们首要目标是确保其能在用户输入发生时立刻运行。在上述场景中，一旦文本编辑器被唤醒，CFS 注意到给它的处理器使用比是 50%，但是其实它却用得少之又少。特别是，CFS 发现文本编辑器比视频解码器运行的时间短得多。这种情况下，为了兑现让所有进程能公平分享处理器的承诺，它会立刻抢占视频解码程序，让文本编辑器投入运行。文本编辑器运行后，立即处理了用户的击键输入后，又一次进入睡眠等待用户下一次输入。因为文本编辑器并没有消费掉承诺给它的 50% 处理器使用比，因此情况依旧，CFS 总是会毫不犹豫地让文本编辑器在需要时被投入运行，而让视频处理程序只能在剩下的时刻运行。

4.4 Linux 调度算法

在前面内容中，我们抽象地讨论了进程调度原理，只是偶尔提及 Linux 如何把给定的理论应用到实际中。在已有的调度原理基础上，我们进一步探讨具有 Linux 特色的进程调度程序。

4.4.1 调度器类

Linux 调度器是以模块方式提供的，这样做的目的是允许不同类型的进程可以有针对性地选择调度算法。

这种模块化结构被称为调度器类（scheduler classes），它允许多种不同的可动态添加的调度算法并存，调度属于自己范畴的进程。每个调度器都有一个优先级，基础的调度器代码定义在 kernel/sched.c 文件中，它会按照优先级顺序遍历调度类，拥有一个可执行进程的最高优先级的调度器类胜出，去选择下面要执行的那一个程序。

完全公平调度（CFS）是一个针对普通进程的调度类，在 Linux 中称为 SCHED_NORMAL（在 POSIX 中称为 SCHED_OTHER），CFS 算法实现定义在文件 kernel/sched_fair.c 中。本节下面的内容将重点讨论 CFS 算法——该内容对于所有 2.6.23 以后的内核版本意义非凡。另外，我们将在 4.4.2 小节讨论实时进程的调度类。

4.4.2 Unix 系统中的进程调度

在讨论公平调度算法前，我们必须首先认识一下传统 Unix 系统的调度过程。正如前面所述，现代进程调度器有两个通用的概念：进程优先级和时间片。时间片是指进程运行多少时间，进程一旦启动就会有一个默认时间片。具有更高优先级的进程将运行得更频繁，而且（在多数系统上）也会被赋予更多的时间片。在 Unix 系统上，优先级以 nice 值形式输出给用户空间。这点听起来简单，但是在现实中，却会导致许多反常的问题，我们下面具体讨论。

第一个问题，若要将 nice 值映射到时间片，就必然需要将 nice 单位值对应到处理器的绝对时间。但这样做将导致进程切换无法最优化进行。举例说明，假定我们将默认 nice 值（0）分配给一个进程——对应的是一个 100ms 的时间片；同时再分配一个最高 nice 值（+20，最低的优先级）给另一个进程——对应的时间片是 5ms。我们接着假定上述两个进程都处于可运行状态。那么默认优先级的进程将获得 20/21（105ms 中的 100ms）的处理器时间，而低优先级的进程会获得 1/21（105ms 中的 5ms）的处理器时间。我们本可以选择任意数值用于本例子中，但这个分配值正好是最具说服力的，所以我们选择它。现在，我们看看如果运行两个同等低优先级的进程情况将如何。我们是希望它们能各自获得一半的处理器时间，事实上也确实如此。但是任何一个进程每次仅仅只能获得 5ms 的处理器时间（10ms 中各占一半）。也就是说，相比刚才例子中 105ms 内进行一次上下文切换，现在则需要在 10ms 内继续进行两次上下文切换。类推，如果是两个具有普通优先级的进程，它们同样会每个获得 50% 处理器时间，但是是在 100ms 内各获得一半。显然，我们看到这些时间片的分配方式并不很理想：它们是给定 nice 值到时间片映射与进程运行优先级混合的共同作用结果。事实上，给定高 nice 值（低优先级）的进程往往是后台进程，且多是计算密集型；而普通优先级的进程则更多是前台用户任务。所以这种时间片分配方式显然是和初衷背道而驰的。

第二个问题涉及相对 nice 值，同时和前面的 nice 值到时间片映射关系也脱不了干系。假设我们有两个进程，分别具有不同的优先级。第一个假设 nice 值只是 0，第二个假设是 1。它们将被分别映射到时间片 100ms 和 95ms（O(1) 调度算法确实这么干了）。它们的时间片几乎一样，其差别微乎其微。但是如果我们的进程分别赋予 18 和 19 的 nice 值，那么它们则分别被映射为 10ms 和 5ms 的时间片。如果这样，前者相比后者获得了两倍的处理器时间！不过 nice 值通常都使用相对值（nice 系统调用是在原值上增加或减少，而不是在绝对值上操作），也就是说：“把进程的 nice 值减小 1”所带来的效果极大地取决于其 nice 的初始值。

第三个问题，如果执行 nice 值到时间片的映射，我们需要能分配一个绝对时间片，而且这个绝对时间片必须能在内核的测试范围内。在多数操作系统中，上述要求意味着时间片必须是定时器节拍的整数倍（请先参看第 11 章“定时器和时间测量”关于时间的讨论）。但这么做必然会引发了几个问题。首先，最小时间片必然是定时器节拍的整数倍，也就是 10ms 或者 1ms 的倍数。其次，系统定时器限制了两个时间片的差异：连续的 nice 值映射到时间片，其差别范围多至 10ms 或者少则 1ms。最后，时间片还会随着定时器节拍改变（如果这里所讨论的定时器节拍对你来说很陌生，快去先看看第 11 章再说。因为这点正是引入 CFS 的唯一原因）。

第四个问题也是最后一个关于基于优先级的调度器为了优化交互任务而唤醒相关进程的问

题。这种系统中，你可能为了进程能更快地投入运行，而去对新要唤醒的进程提升优先级，即便它们的时间片已经用尽了。虽然上述方法确实能提升不少交互性能，但是一些例外情况也有可能发生，因为它同时也给某些特殊的睡眠/唤醒用例一个玩弄调度器的后门，使得给定进程打破公平原则，获得更多处理器时间，损害系统中其他进程的利益。

上述问题中的绝大多数都可以通过对传统 Unix 调度器进行改造解决，虽然这种改造修改不小，但也并非是结构性调整。比如，将 nice 值呈几何增加而非算数增加的方式解决第二个问题；采用一个新的度量机制将从 nice 值到时间片的映射与定时器节拍分离开来，以此解决第三个问题。但是这些解决方案都回避了实质问题——即分配绝对的时间片引发的固定的切换频率，给公平性造成了很大变数。CFS 采用的方法是对时间片分配方式进行根本性的重新设计（就进程调度器而言）：完全摒弃时间片而是分配给进程一个处理器使用比重。通过这种方式，CFS 确保了进程调度中能有恒定的公平性，而将切换频率置于不断变动中。

4.4.3 公平调度

CFS 的出发点基于一个简单的理念：进程调度的效果应如同系统具备一个理想中的完美多任务处理器。在这种系统中，每个进程将能获得 $1/n$ 的处理器时间—— n 是指可运行进程的数量。同时，我们可以调度给它们无限小的时间周期，所以在任何可测量周期内，我们给予 n 个进程中每个进程同样多的运行时间。举例来说，假如我们有两个运行进程，在标准 Unix 调度模型中，我们先运行其中一个 5ms，然后再运行另一个 5ms。但它们任何一个运行时都将占有 100% 的处理器。而在理想情况下，完美的多任务处理器模型应该是这样的：我们能在 10ms 内同时运行两个进程，它们各自使用处理器一半的能力。

当然，上述理想模型并非现实，因为我们无法在一个处理器上真的同时运行多个进程。而且如果每个进程运行无限小的时间周期也是不高效的——因为调度时进程抢占会带来一定的代价：将一个进程换出，另一个换入本身有消耗，同时还会影响到缓存的效率。因此虽然我们希望所有进程能只运行一个非常短的周期，但是 CFS 充分考虑了这将带来的额外消耗，实现中首先要确保系统性能不受损失。CFS 的做法是允许每个进程运行一段时间、循环轮转、选择运行最少的进程作为下一个运行进程，而不再采用分配给每个进程时间片的做法了，CFS 在所有可运行进程总数基础上计算出一个进程应该运行多久，而不是依靠 nice 值来计算时间片。nice 值在 CFS 中被作为进程获得的处理器运行比的权重：越高的 nice 值（越低的优先级）进程获得更低的处理器使用权重，这是相对默认 nice 值进程的进程而言的；相反，更低的 nice 值（越高的优先级）的进程获得更高的处理器使用权重。

每个进程都按其权重在全部可运行进程中所占比例的“时间片”来运行，为了计算准确的时间片，CFS 为完美多任务中的无限小调度周期的近似值设立了一个目标。而这个目标称作“目标延迟”，越小的调度周期将带来越好的交互性，同时也更接近完美的多任务。但是你必须承受更高的切换代价和更差的系统总吞吐能力。让我们假定目标延迟值是 20ms，我们有两个同样优先级的可运行任务（无论这些任务的优先级是多少）。每个任务在被其他任务抢占前运行 10ms，如果我们有 4 个这样的任务，则每个只能运行 5ms。进一步设想，如果有 20 个这样的任务，那么每个仅仅只能获得 1ms 的运行时间。

你一定注意到了，当可运行任务数量趋于无限时，它们各自所获得的处理器使用比和时间片都将趋于 0。这样无疑造成了不可接受的切换消耗。CFS 为此引入每个进程获得的时间片底线，这个底线称为最小粒度。默认情况下这个值是 1ms。如此一来，即便是可运行进程数量趋于无穷，每个最少也能获得 1ms 的运行时间，确保切换消耗被限制在一定范围内。（敏锐的读者会注意到假如在进程数量变得非常多的情况下，CFS 并非一个完美的公平调度，因为这时处理器时间片再小也无法突破最小粒度。的确如此，尽管修改过的公平队列方法确实能提高这方面的公平性，但是 CFS 的算法本身其实已经决定在这方面做出折中了。但还好，因为通常情况下系统中只会有几百个可运行进程，无疑，这时 CFS 是相当公平的。）

现在，让我们再来看看具有不同 nice 值的两个可运行进程的运行情况——比如一个具有默认 nice 值 (0)，另一个具有的 nice 值是 5。这些不同的 nice 值对应不同的权重，所以上述两个进程将获得不同的处理器使用比。在这个例子中，nice 值是 5 的进程的权重将是默认 nice 进程的 1/3。如果我们的目标延迟是 20ms，那么这两个进程将分别获得 15ms 和 5ms 的处理器时间。再比如我们的两个可运行进程的 nice 值分别是 10 和 15，它们分配的时间片将是多少呢？还是 15 和 5ms！可见，绝对的 nice 值不再影响调度决策：只有相对值才会影响处理器时间的分配比例。

总结一下，任何进程所获得的处理器时间是由它自己和其他所有可运行进程 nice 值的相对差值决定的。nice 值对时间片的作用不再是算数加权，而是几何加权。任何 nice 值对应的绝对时间不再是一个绝对值，而是处理器的使用比。CFS 称为公平调度器是因为它确保给每个进程公平的处理器使用比。正如我们知道的，CFS 不是完美的公平，它只是近乎完美的多任务。但是它确实多进程环境下，降低了调度延迟带来的不公平性。

4.5 Linux 调度的实现

在讨论了采用 CFS 调度算法的动机和其内部逻辑后，我们现在可以开始具体探索 CFS 是如何得以实现的。其相关代码位于文件 `kernel/sched_fair.c` 中，我们将特别关注其四个组成部分：

- 时间记账
- 进程选择
- 调度器入口
- 睡眠和唤醒

4.5.1 时间记账

所有的调度器都必须对进程运行时间做记账。多数 Unix 系统，正如我们前面所说，分配一个时间片给每一个进程。那么当每次系统时钟节拍发生时，时间片都会被减少一个节拍周期。当一个进程的时间片被减少到 0 时，它就会被另一个尚未减到 0 的时间片可运行进程抢占。

1. 调度器实体结构

CFS 不再有时间片的概念，但是它也必须维护每个进程运行的时间记账，因为它需要确保每个进程只在公平分配给它的处理器时间内运行。CFS 使用调度器实体结构（定义在文件 `<linux/sched.h>` 的 `struct_sched_entity` 中）来追踪进程运行记账：


```

struct sched_entity {
    struct load_weight    load;
    struct rb_node        run_node;
    struct list_head      group_node;
    unsigned int          on_rq;
    u64                   exec_start;
    u64                   sum_exec_runtime;
    u64                   vruntime;
    u64                   prev_sum_exec_runtime;
    u64                   last_wakeup;
    u64                   avg_overlap;
    u64                   nr_migrations;
    u64                   start_runtime;
    u64                   avg_wakeup;
    /* 这里省略了很多统计变量，只有在设置了 CONFIG_SCHEDSTATS 时才启用这些变量 */
};

```

调度器实体结构作为一个名为 `se` 的成员变量，嵌入在进程描述符 `struct task_struct` 内。我们已经在第 3 章讨论过进程描述符。

2. 虚拟实时

`vruntime` 变量存放进程的虚拟运行时间，该运行时间（花在运行上的时间和）的计算是经过所有可运行进程总数的标准化（或者说是被加权的）。虚拟时间是以 `ns` 为单位的，所以 `vruntime` 和定时器节拍不再相关。虚拟运行时间可以帮助我们逼近 CFS 模型所追求的“理想多任务处理器”。如果我们真有这样一个理想的处理器，那么我们就不再需要 `vruntime` 了。因为优先级相同的所有进程的虚拟运行时都是相同的——所有任务都将接收到相等的处理器份额。但是因为处理器无法实现完美的多任务，它必须依次运行每个任务。因此 CFS 使用 `vruntime` 变量来记录一个程序到底运行了多长时间以及它还应该再运行多久。

定义在 `kernel/sched_fair.c` 文件中的 `update_curr()` 函数实现了该记账功能：

```

static void update_curr(struct cfs_rq *cfs_rq)
{
    struct sched_entity *curr = cfs_rq->curr;
    u64 now = rq_of(cfs_rq)->clock;
    unsigned long delta_exec;

    if (unlikely(!curr))
        return;
    /* 获得从最后一次修改负载后当前任务所占用的运行总时间（在 32 位系统上这不会溢出） */
    delta_exec = (unsigned long)(now - curr->exec_start);
    if (!delta_exec)
        return;
    __update_curr(cfs_rq, curr, delta_exec);
    curr->exec_start = now;

    if (entity_is_task(curr)) {
        struct task_struct *curtask = task_of(curr);

```